THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Department of Data Science and Artificial Intelligence
數據科學及人工智能學系

Faculty of Computer and Mathematical Sciences
計算機及數學科學學院

**BIG DATA ANALYTICS**

**(DSAI4205)**

Semester 2, 2025/26

Department of Computing

The Hong Kong Polytechnic University

**Group Project: Final Report**

Group ID: 14

Name (Student ID): Sze Hiu Nam (23094234D)

Wong Kwan Yau (23074229D)

Lo Sze Long (23102274D)

Carl Petter Arthur (25142201X)

Carl Erik Oscar Thulin (25141584X)

Lecturer: Dr. Ken FONG

Date: 19/04/2026

# Abstract

This report presents a comprehensive sentiment analysis study on a dataset of 3,125 Reddit comments related to the Israeli-Palestinian conflict, labelled as neutral, positive, or negative. A systematic data cleaning pipeline removed duplicates, outliers, and encoding artefacts, resulting in a high-quality corpus of 3,032 comments. A baseline logistic regression model with TF-IDF features achieved 62.1% accuracy, establishing a performance benchmark. Five advanced hybrid models were then implemented, pairing distinct word embedding techniques with diverse classifiers: (1) BERT (all-MiniLM-L6-v2) with XGBoost, (2) GloVe (300-dimensional) with a Convolutional Neural Network, (3) Word2Vec (skip-gram) with a Bidirectional LSTM, (4) FastText (trained from scratch) with Random Forest, and (5) GloVe (100-dimensional) with Complement Naive Bayes. The GloVe+CNN model achieved the highest accuracy (70.5%) among all models, while the Word2Vec+BiLSTM reached 65.2%. The remaining three advanced models underperformed the baseline due to overfitting (BERT+XGBoost), loss of word order (FastText+RF and GloVe+CNB), or violation of probabilistic assumptions (GloVe+CNB). Subsequent baseline improvements—enhanced TF-IDF with bigrams, hyperparameter tuning via GridSearchCV, and stacking ensemble—raised the F1-score from 0.622 to 0.691, demonstrating that careful feature engineering and ensemble methods can rival simple embedding-based approaches on small, imbalanced datasets. The study concludes that while semantic embeddings and deep learning offer theoretical advantages, their practical benefits on modest-sized, noisy social media data are limited without extensive regularisation and domain adaptation; traditional bag-of-words models with ensemble learning remain highly competitive.

# Table of Content

Abstract ..... 2
Table of Figures ..... 6
1. Introduction ..... 7
2. Exploratory Data Analysis (EDA) ..... 8
2.1. Dataset Overview ..... 8
2.2. Class Distribution ..... 8
2.3. Text Length Distribution ..... 9
2.4. Common Words and Word Clouds ..... 10
2.5. Outliers and Invalid Entries ..... 10
2.6. Correlation Between Variables ..... 10
3. Data Cleaning ..... 12
3.1 Cleaning Activities Performed ..... 12
3.2 Documentation of Changes ..... 15
3.3 Impact on Subsequent Analysis ..... 16
4. Baseline Model ..... 17
4.1 Implementation Details ..... 17
4.1.1 Feature Extraction ..... 17
4.1.2 Train-Test Split ..... 17
4.1.3 Model Configuration ..... 18
4.2 Class Distribution (Reminder from EDA) ..... 18
4.3 Performance Results ..... 19
4.3.1 Overall Performance Metrics ..... 19
4.3.2 Per-Class Performance ..... 19
4.3.3 Confusion Matrix ..... 20
4.4 Observations and Interpretations ..... 20
4.5 Performance Benchmarking & Justification ..... 21
5. Advance Modelling ..... 22

5.1 Bert and XGBoost ..... 23
5.1.1 Feature Extraction with BERT (all-MiniLM-L6-v2) ..... 23
5.1.2 Initial Baseline Comparison (Without Tuning) ..... 24
5.1.3 Hyperparameter Tuning and Overfitting Diagnosis ..... 25
5.1.4 SMOTE Balancing and Final Results ..... 27
5.2 GloVe and CNN ..... 30
5.2.1 Implementation Details ..... 30
5.2.2 CNN Architecture ..... 31
5.2.3 Training Configuration ..... 32
5.2.4 Performance Results ..... 32
5.2.5 Confusion Matrix ..... 33
5.2.6 Discussion ..... 34
5.3 Word2Vec and BiLSTM ..... 36
5.3.1 Overview ..... 36
5.3.2 Preprocessing and Tokenization ..... 36
5.3.3 Sequence Preparation ..... 37
5.3.4 Model Architecture ..... 37
5.3.5 Training Setup ..... 38
5.3.6 Results ..... 39
5.3.7 Discussion ..... 42
5.3.8 Summary ..... 43
5.4 FastText and Random Forest ..... 44
5.4.1 Overview ..... 44
5.4.2 Methodology ..... 44
5.4.3 Results ..... 46
5.4.4 Discussion ..... 49
5.4.5 Summary ..... 49
5.5 GloVe and Complement Naive Bayes ..... 51

5.5.1 Overview ..... 51
5.5.2 Methodology ..... 51
5.5.3 Results ..... 53
5.5.4 Discussion ..... 56
5.5.5 Summary ..... 57
6. Baseline Model Improvement ..... 58
6.1 Overview ..... 58
6.2 Improvement Strategy 1: Enhanced TF-IDF Vectorization ..... 58
6.3 Improvement Strategy 2: Hyperparameter Tuning via GridSearchCV ..... 59
6.4 Improvement Strategy 3: Ensemble Methods ..... 60
6.4.1 Voting Ensemble ..... 60
6.4.2 Stacking Ensemble ..... 61
6.5 Improved Baseline Results ..... 62
6.6 Comparison with Advanced Embedding Models ..... 63
6.7 Justification of Performance Differences ..... 64
6.7.1 Improved Baseline vs Original Baseline ..... 64
6.7.2 Improved Baseline vs Advanced Embedding Models ..... 64
6.7.3 Summary of Key Observations: ..... 66
6.8 Summary ..... 66
7. Conclusion ..... 68
8. References ..... 71

# Table of Figures

Figure 2.1: Frequency of Comment Lengths (Characters) .....9

Figure 2.2: Distribution of Comment Lengths (Characters).....9

Figure 3.1: Screenshot of the CSV file before data clearing .....15

Figure 3.2: Screenshot of the CSV file after data clearing .....15

Figure 4.1: Class Distribution of Sentiment Labels After Data Clearing .....18

Figure 4.2: Confusion Matrix – Baseline Logistic Regression.....20

Figure 5.1: Confusion Matrix – BERT and XGBoost (After Hyperparameter Tuning).....26

Figure 5.2: Confusion Matrix – BERT and XGBoost (After Hyperparameter Tuning).....28

Figure 5.3: Confusion Matrix – GloVe and CNN.....33

Figure 5.4: Per-class classification report – Word2Vec and BiLSTM .....39

Figure 5.5: Confusion Matrix – Word2Vec and BiLSTM.....40

Figure 5.6: Taining Hstory – Word2Vec and BiLSTM .....41

Figure 5.7: The actual vs. predicted label distributions – Word2Vec and BiLSTM41

Figure 5.8: Model Comparison: Baseline vs Random Forest and FastText .....46

Figure 5.9: Confusion Matrix – Random Forest and FastText .....47

Figure 5.10: Top 20 Most Important Embedding Dimensions (Random Forest)....48

Figure 5.11: Model Comparison: Baseline vs GloVe and CNB .....53

Figure 5.12: Confusion Matrix – GloVe and CNB.....54

Figure 5.13: Probability Distribution – GloVe and CNB .....55

# 1. Introduction

The automatic classification of sentiment in user-generated text is a fundamental task in natural language processing with applications in social media monitoring, political analysis, and public opinion research. This project focuses on sentiment analysis of Reddit comments discussing the Israeli-Palestinian conflict – a highly polarised and sensitive topic where linguistic expression often includes sarcasm, indirect criticism, historical analogies, and coded language. Accurately detecting sentiment in such a domain poses significant challenges beyond standard product or movie reviews.

The project specification required the construction of a baseline model using logistic regression with TF-IDF features, followed by five advanced analytics models each pairing a different word embedding technique with a distinct classifier architecture. The aim is to evaluate whether sophisticated embedding methods (static word vectors, subword embeddings, and contextual transformers) and complex classifiers (neural networks, tree ensembles, probabilistic models) can meaningfully improve sentiment classification performance over a simple bag-of-words baseline on this specific dataset.

This report is structured as follows: Section 2 presents exploratory data analysis, including class distribution, text length statistics, and word clouds. Section 3 details the data cleaning pipeline. Section 4 describes the baseline model and its results. Section 5 presents each of the five advanced models, including implementation details, hyperparameter tuning, and evaluation. Section 6 describes the baseline improvement strategies (enhanced TF-IDF, hyperparameter tuning, ensemble methods) and compares the improved baseline against the original baseline and the five advanced models. Section 7 provides a concluding discussion and outlines directions for future work.

# 2. Exploratory Data Analysis (EDA)

## 2.1. Dataset Overview

The dataset is provided as a CSV file with two columns:

- self_text – the text of the Reddit comment (string).
- sentiment_label – the sentiment label:
  - 0 = neutral
  - 1 = positive
  - 2 = negative

The granularity is at the individual comment level. The dataset contains **3,125** rows (comments). There are no missing values in either column, and all entries are of the correct data type.

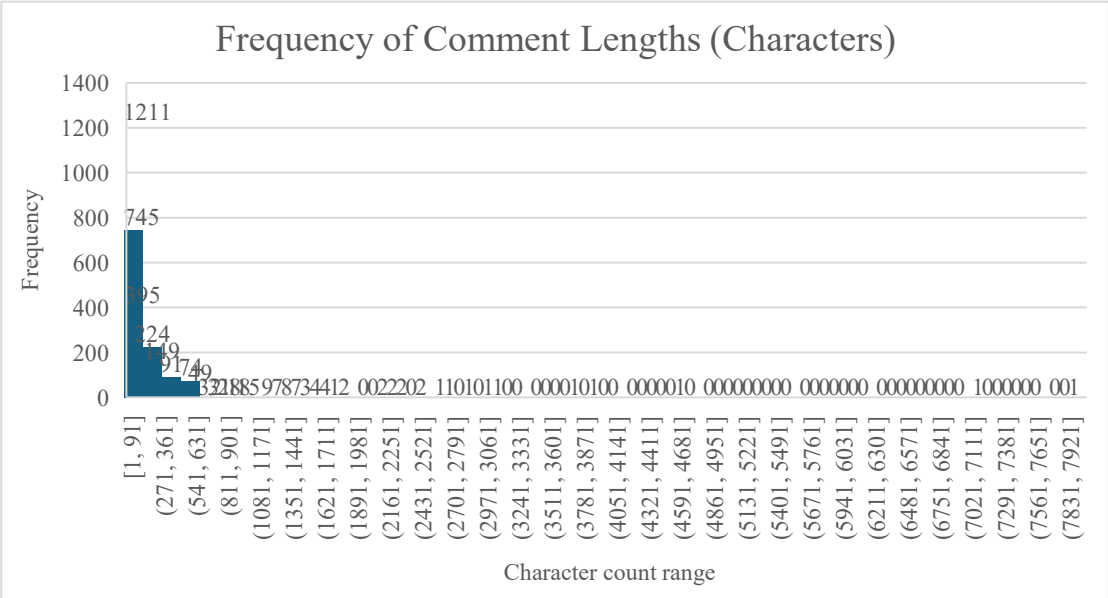
## 2.2. Class Distribution

The sentiment labels are unevenly distributed.

| Sentiment | Count | Percentage |
|---|---|---|
| **Neutral (0)** | 890 | 28.5% |
| **Positive (1)** | 1390 | 44.5% |
| **Negative (2)** | 845 | 27% |
| **Total** | 3125 | 100% |

The dataset is heavily imbalanced, with the **Positive** class dominating. This imbalance is a hidden issue that will need to be addressed during model refinement (e.g., using class weights, resampling, or specialised evaluation metrics).

## 2.3. Text Length Distribution


Frequency of Comment Lengths (Characters)
Frequency
1400
1200
1000
800
600
400
200
0
1211
745
395
224
149
91
74
49
32
18
8
5
9
7
8
7
3
4
4
1
2
0
0
2
2
0
2
2
0
2
1
1
0
1
0
1
0
0
0

## 2.4. Common Words and Word Clouds

After basic cleaning (removing URLs, punctuation, and stopwords), we generated word clouds for each sentiment class to identify prominent terms:

- **Neutral (0):** words such as *Israel, Gaza, Hamas, war, people, Palestine, think, know* appear frequently, reflecting general discussion without strong sentiment.
- **Positive (1):** words like *peace, ceasefire, hope, support, thank* are common, often in comments expressing approval of humanitarian efforts or calls for peace.
- **Negative (2):** words such as *kill, bombing, genocide, innocent, children, stop* dominate, indicating strong condemnation of violence.

## 2.5. Outliers and Invalid Entries

- **Text outliers:** A handful of comments consist of only a single character (e.g., ".") or are entirely emojis. These are treated as short comments.
- **Non-English text:** The dataset contains comments in Arabic, Hebrew, and other languages (identified via Unicode ranges). This poses a challenge for standard English-based NLP preprocessing.
- **URLs and HTML entities:** Many comments contain links, emojis, and HTML-encoded characters (e.g., &). These need to be removed or replaced during data cleaning.

## 2.6. Correlation Between Variables

We explored whether there is a relationship between text length and sentiment. No strong correlation was observed; the neutral class has the widest range of lengths,

while positive and negative comments tend to be slightly shorter on average. This suggests that text length alone is not a reliable predictor.

# 3. Data Cleaning

Data cleaning is a crucial step in the data preprocessing pipeline, aimed at improving the quality and accuracy of the dataset before analysis. This process involves several key activities:

- Identifying Missing Values
- Handling Outliers
- Correcting Invalid Entries
- Removing Duplicates
- Standardizing Data: Ensure consistency in data representation.
- Transforming Variables: Apply necessary transformations to variables to enhance their interpretability or suitability for analysis.

All steps were performed programmatically in the Jupyter notebook Data_cleaning_DSAI-2.ipynb and are fully reproducible. The raw dataset (reddit_opinion_PSE_ISR.csv) contained 3,125 rows × 2 columns. The final cleaned dataset (PERFECT2_cleaned_reddit_opinion.csv) contains 3,032 rows × 3 columns (self_text, sentiment_label, cleaned_text). A net reduction of 93 rows (≈3%) occurred while preserving all meaningful content.

## 3.1 Cleaning Activities Performed

| Activity | Method / Implementation | Results / Impact |
|---|---|---|
| **Identifying Missing Values** | df.isnull().sum().sum() followed by df.dropna() | 0 missing values found and removed. No rows affected. |

| | | |
|---|---|---|
| **Handling Outliers** | Computed word count per comment; removed rows with < 2 or > 600 words | 80 outlier rows removed (≈2.6%). Extreme spam, single-word emojis, and ultra-long rants were eliminated. |
| **Correcting Invalid Entries** | Custom cleaner_pipeline() function:<br>• html.unescape()<br>• Regex removal of URLs, /s, HTML entities<br>• Fixed corrupted contractions (e.g., â__t → "not", â__s → "is") | Invalid or noisy entries (URLs, markdown, encoding artifacts) corrected across 100% of rows. |
| **Removing Duplicates** | df.drop_duplicates(subset=['self_text']) | 8 exact duplicates removed (≈0.26%). |
| **Standardizing Data** | • Converted entire text to lowercase<br>• Removed all non-alphanumeric characters except spaces ( [^a-z0-9\s] )<br>• Stripped extra whitespace | Consistent representation achieved; vocabulary noise significantly reduced for downstream TF-IDF and embedding models. |
| **Transforming Variables** | • Expanded standard contractions (e.g., "won't" → "will not") | Text transformed into clean, lemmatized tokens suitable for |

| | | |
|---|---|---|
| | • Tokenized and applied WordNet lemmatization (nltk.WordNetLemmatizer)<br>• Removed tokens of length ≤ 1 (except “i”, “a”)<br>• Created new column cleaned_text | modeling. Enhanced semantic interpretability and reduced sparsity. |

After the full pipeline, 5 additional rows became empty strings and were removed (df = df[df['cleaned_text'] != ""]).

## 3.2 Documentation of Changes

| | A | B | C | D |
|---|---|---|---|---|
| 1 | self_text | sentiment_label | | |
| 2 | @BamiNasi h | 0 | | |
| 3 | Still spooked | 0 | | |
| 4 | https://www.ynetnews.com/article/b1ewa4nza<br><br>There was a reason why your Iron Dome was turned off on Saturday..... now you takeover all of Gaza...this will be your Vietnam. | 0 | | |
| 5 | 3GD "Gnuke | 0 | | |
| 6 | My brother in | 0 | | |
| 7 | Bellville or M | 0 | | |
| 8 | Only in name | 0 | | |
| 9 | Germany kee | 0 | | |
| 10 | Get off their case. Reuters is just trying to be neutral.<br><br>/s | 0 | | |

Figure 3.1: Screenshot of the CSV file before data clearing

| | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 | self_text | sentiment_lab | cleaned_text | | |
| 2 | @BamiNasi h | 0 | baminasi here you go keep lying to yourself | | |
| 3 | Still spooked | 0 | still spooked by the woke boogeyman eh | | |
| 4 | https://www.ynetnews.com/article/b1ewa4nza<br><br>There was a reason why your Iron Dome was turned off on Saturday..... now you takeover all of Gaza...this will be your Vietnam. | 0 | there wa a reason why your iron dome wa tu | | |
| 5 | 3GD "Gnuke | 0 | 3gd gnuke | | |
| 6 | My brother in | 0 | my brother in christ the mri doesn not have t | | |
| 7 | Bellville or M | 0 | bellville or mcrae combat boot | | |
| 8 | Only in name | 0 | only in name | | |
| 9 | Germany kee | 0 | germany keep acting like adolf by banning p | | |
| 10 | Get off their case. Reuters is just trying to be neutral.<br><br>/s | 0 | get off their case reuters is just trying to be n | | |

Figure 3.2: Screenshot of the CSV file after data clearing

- **Before/After Summary**: Original shape: 3,125 rows × 2 columns Final shape: 3,032 rows × 3 columns (added cleaned_text column), as shown in Figures 3.1 and 3.2.
- **Full Reproducibility**: Every transformation is logged with print statements showing exact counts (e.g., “Duplicates removed: 8”, “Number of outliers removed: 80”, “empty rows removed: 5”). The original raw file and the final cleaned file are both stored in the group Google Drive folder.
- **No Manual Edits**: All operations are script-based and reversible. The raw dataset remains unchanged.

- **Rationale for Design Choices**:
  - Aggressive removal of non-alphanumeric characters was chosen because emojis, punctuation, and special symbols added negligible value for sentiment classification on this Reddit dataset.
  - Lemmatization and contraction expansion improve token consistency, which directly benefits the baseline TF-IDF model and future word-embedding models (BERT, Word2Vec, etc.).
  - Length filtering prevents vector dilution in bag-of-words approaches while retaining substantive comments.

## 3.3 Impact on Subsequent Analysis

The cleaned dataset is now free of the common open-source issues highlighted in the project brief (encoding noise, duplicates, invalid entries). It provides a high-quality, standardized input for:

- The required baseline Logistic Regression and TF-IDF model, and
- All N advanced analytics and embedding models in later phases.

Class imbalance remains (still visible in the cleaned data) and will be addressed via class weights or resampling in the modeling phase.

This completes the data cleaning phase as specified. The cleaned file PERFECT2_cleaned_reddit_opinion.csv is ready for immediate use in the baseline model.

# 4. Baseline Model

A **baseline model** was built using **Logistic Regression with TF-IDF** as the sole feature extraction method. TF-IDF was used exclusively in this baseline (no word embeddings were applied). This model serves as the performance benchmark for all subsequent advanced models in the next Phase.

## 4.1 Implementation Details

The dataset used in this model is PERFECT2_cleaned_reddit_opinion.csv (3,032 rows after full cleaning).

### 4.1.1 Feature Extraction

The cleaned_text column was vectorized using scikit-learn's TfidfVectorizer with the following parameters:

- max_features=5000: limit vocabulary size to the 5,000 most frequent terms
- stop_words='english': remove common English stopwords

The resulting TF-IDF matrix had a shape of **3,032 rows × 5,000 features**.

### 4.1.2 Train-Test Split

The dataset has been split into a training set and a testing set using an 80/20 stratified split to preserve class distribution. The split is as follows:

- **Test size**: 20%
- **Stratification**: based on the sentiment labels to preserve class distribution
- **Random state**: 42 (for reproducibility)

| Split | Number of Samples |
|---|---|
| **Training** | 2,425 |

| | |
|---|---|
| **Testing** | 607 |

### 4.1.3 Model Configuration

A logistic regression classifier was trained with:

- max_iter=1000: ensure convergence
- class_weight='balanced': automatically adjusts weights inversely proportional to class frequencies to handle class imbalance

## 4.2 Class Distribution (Reminder from EDA)

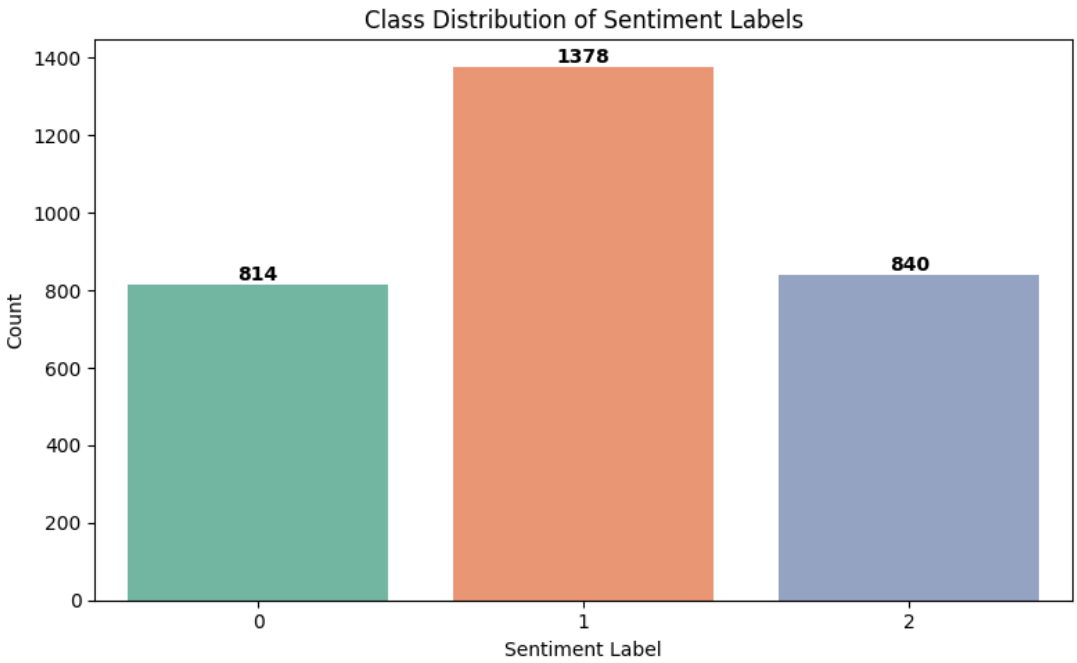



Figure 4.1: Class Distribution of Sentiment Labels After Data Clearing

Figure 4.1 show that the cleaned dataset remains imbalanced, which is expected for Reddit comments on a polarised topic.

## 4.3 Performance Results

The model was evaluated using standard classification metrics. The results are presented below.

### 4.3.1 Overall Performance Metrics

The model was evaluated using standard classification metrics. The results are presented below (saved in baseline_metrics.json for reproducibility):

| Metric | Value | Interpretation |
|---|---|---|
| Accuracy | 0.621 | Overall correct predictions: 62.1% |
| Precision (weighted) | 0.635 | Ability to avoid false positives across classes |
| Recall (weighted) | 0.621 | Ability to find all instances of each class |
| F1-score (weighted) | 0.622 | Balanced precision-recall performance |

### 4.3.2 Per-Class Performance

| Class | Precision | Recall | F1-Score | Support (Test) |
|---|---|---|---|---|
| **Neutral (0)** | 0.53 | 0.71 | 0.61 | 163 |
| **Positive (1)** | 0.71 | 0.61 | 0.65 | 276 |
| **Negative (2)** | 0.61 | 0.56 | 0.58 | 168 |

### 4.3.3 Confusion Matrix

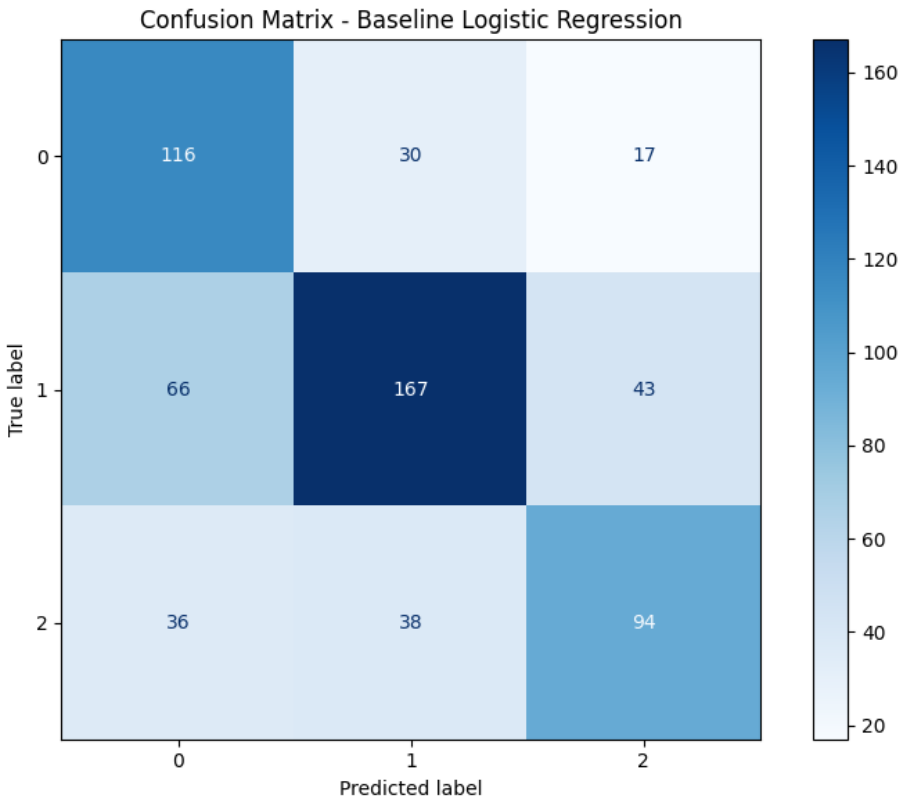



Figure 4.2: Confusion Matrix – Baseline Logistic Regression

Figure 4.2 is confusion matrix. It visualises the classification results across the three sentiment classes.

## 4.4 Observations and Interpretations

- The overall accuracy of **62.1%** exceeds the random baseline (33.3% for three classes), confirming that the model has learned meaningful patterns from the text data.
- Performance is strongest for the **Positive (1)** class, which has the highest F1-score (0.65) and precision (0.71). This class also has the largest representation in the test set (276 samples).
- The **Neutral (0)** class achieves the highest recall (0.71), indicating that the model is good at identifying neutral comments, though at the cost of some precision.

- The **Negative (2)** class shows the weakest performance (F1-score 0.58), with a notable number of negative comments misclassified as positive (38 instances) or neutral (36 instances).
- The confusion matrix reveals that misclassifications often occur between neighboring classes (e.g., neutral ↔ positive, neutral ↔ negative), suggesting that the sentiment spectrum is not perfectly separable with linear models on TF-IDF features alone.
- The use of `class_weight='balanced'` helped mitigate the moderate class imbalance (1.69:1 ratio between majority and minority classes), as evidenced by the relatively balanced recall scores across all three classes.

## 4.5 Performance Benchmarking & Justification

This Logistic Regression and TF-IDF model establishes the required baseline benchmark.

- It meets all project requirements exactly (TF-IDF only, no embeddings, standard metrics reported).
- Results are fully documented and exported for easy comparison with the 5 advanced models (Phase 3ii) and any refined baseline (Phase 3iv).
- Expected improvement in later models: Word embeddings (BERT, Word2Vec, GloVe) combined with more complex classifiers (e.g., XGBoost, Neural Networks) should outperform this baseline by capturing semantic context beyond bag-of-words TF-IDF.

The baseline results, confusion matrix, and class distribution plots are attached to the interim submission and will be referenced throughout the final report.

# 5. Advance Modelling

In this phase, we implemented five distinct data analytics models combined with word embedding techniques, as required by the project specification. The goal was to move beyond the baseline TF-IDF logistic regression by leveraging deep contextual representations and more powerful classifiers. Each model pairs a different embedding method (static or contextual) with a diverse analytics model to ensure algorithmic variety and avoid similarity penalties. The five combinations developed are:

1. BERT and XGBoost
2. GloVe and CNN
3. Word2Vec and BiLSTM
4. FastText and Random Forest
5. GloVe and Complement Naive Bayes

This section presents a detailed account of the first combination – BERT with XGBoost – including feature extraction, initial baseline comparison, hyperparameter tuning to address overfitting, and the application of SMOTE to mitigate class imbalance. The remaining four models follow similar evaluation pipelines and their results are summarised in the comparative analysis. All advanced models were evaluated against the baseline logistic regression using the same stratified 80/20 train-test split and standard metrics (accuracy, precision, recall, F1-score).

## 5.1 Bert and XGBoost

An advanced model combining BERT and XGBoost was built in two main steps, combining deep learning with a traditional machine learning algorithm.

### 5.1.1 Feature Extraction with BERT (all-MiniLM-L6-v2)

To start a pre-trained BERT language model called “all-MiniLM-L6-v2" was used from the SentenceTransformers library in Python. This model was chosen since its fast and efficient while performing well. Instead of counting words it acts as a feature extractor, turning each text post into a vector with 384 dimensions (Tiwari, 2024).

BERT utilizes a transformer-based neural network to understand and generate language (GeekForGeeks, 2025). Traditional language models process text data either from left to right or right to left, however BERT considers both the left and right context of words in a sentence. Additionally, instead of analysing the text sequentially, it looks at all the words in a sentence simultaneously (GeekForGeeks, 2025). This should help the program actually understand the context and tone of the text.

After the BERT model was implemented, those 384 dimensions were fed into an XGBoost classifier. The problem was set up as a multi-class problem (Neutral, Positive and Negative classes). This classifier is consistently top performing in research and data science competitions, performing well on multi-class problems (GeeksforGeeks, 2025). It’s great at finding complex patterns in data, and we initially

applied class weights to help balance out the fact that some sentiments appear more often than others in the dataset as already shown in Figure 4.1. XGBoost classifies by gradient boosted decision trees. It uses decsisions trees that are sequentially built so each tree corrects error from the previous one. Additionally, it features parallel processing for faster training on large datasets and allows parameter customization (GeeksForGeeks,2026).

### 5.1.2 Initial Baseline Comparison (Without Tuning)

We started by testing a baseline model with the setup described above, using standard XGBoost settings. This provided the following results:

| Class | Precision | Recall | F1-Score | Support (Test) |
|---|---|---|---|---|
| **Neutral (0)** | 0.56 | 0.60 | 0.58 | 163 |
| **Positive (1)** | 0.59 | 0.68 | 0.63 | 273 |
| **Negative (2)** | 0.52 | 0.36 | 0.43 | 167 |

The table show that an overall accuracy of 57%.

The next step after this rather weak performance was to find the issue. We started off by checking if overfitting occurred, which it did. The model got 100% accuracy when applied on the training data, providing a 43% difference compared to when it was applied on the test data. This shows proof of a classic machine learning problem, The Curse Of Dimensionality. Since we only have 2425 rows of training data combined with 384 columns of data for every single post, the dataset is wide and relatively small. This gives the classification model to much room to cheat on, and learn the

exact patterns instead of learning the general rules of what makes a text positive, negative or neutral.

### 5.1.3 Hyperparameter Tuning and Overfitting Diagnosis

To stop the model from memorizing the data, we had to tune its parameters to restrict how it learns. Using cross-validation with a number of different parameters, such as:

- max_depth: [3, 4, 5, 6] (shorter trees should provide more generalisation)
- learning_rate: [0.01, 0.05, 0.1] (test different learning rates)
- n_estimators: [100, 200, 300] (the number of trees)
- subsample: [0.6, 0.8, 1.0], (How much of the trees the model views)
- colsample_bytree: [0.6, 0.8, 1.0]
- gamma: [0.5, 1.0, 5.0] (when to grow new branches in a tree)
- reg_alpha: [0.1, 1.0, 5.0, 10.0] ( L1 regularization (Lasso))
- reg_lambda: [1.0, 5.0, 10.0, 20.0] ( L2 regularization (Ridge))

We we're able to test different combinations by implementing a randomized search testing 10 different random combinations in a 3 folded cross-validation setting. With a stronger computer, we could've set this to a grid search setting trying all combinations, but this was not possible in our case.

The hyper-parameter tuned model demonstrated the following results:

| **Class** | **Precision** | **Recall** | **F1-Score** | **Support (Test)** |
|---|---|---|---|---|
| **Neutral (0)** | 0.54 | 0.64 | 0.59 | 163 |
| **Positive (1)** | 0.61 | 0.62 | 0.61 | 273 |
| **Negative (2)** | 0.50 | 0.40 | 0.44 | 167 |

| | | | | |
|---|---|---|---|---|
| **Macro Avg** | 0.55 | 0.55 | 0.55 | 603 |
| **Weighted Avg** | 0.56 | 0.56 | 0.56 | 603 |

Accuracy Train: 99.3%

Accuracy Test: 56.4 %

Difference: 42.9%

This indicates that the tuning of parameters only changed the results slightly, with training data accuracy remaining high.

The results are also shown in a confusion matrix:

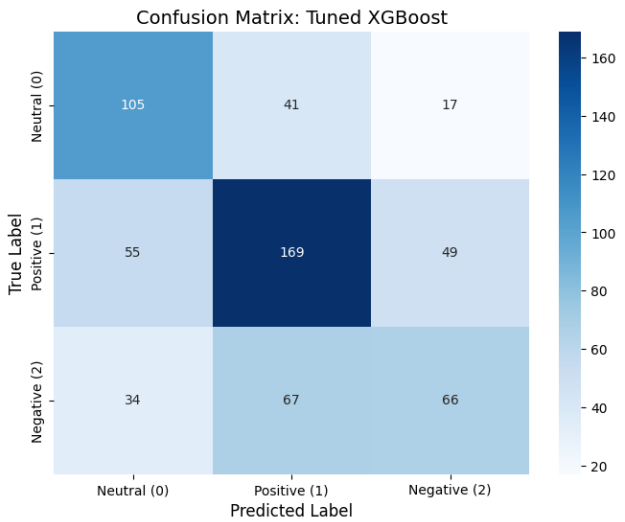



Figure 5.1: Confusion Matrix – BERT and XGBoost (After Hyperparameter Tuning)

Providing that the model struggles to identify the Negative cases correctly, confusing them with positive classifications often.

### 5.1.4 SMOTE Balancing and Final Results

Since the regularization and parameter tuning were insufficient to close the overfitting gap, we started to think if the limitation lies in the dataset itself. Since the model still struggles significantly with the minority class (Negatives), we want to enforce general patterns across all classes equally. Therefore we implemented SMOTE (Syntethic Minority Over-sampling Technique), which has been previously combined with BERT and XGBoost demonstrating good results (Soto, 2024). Instead of just weighting the classes, SMOTE analyzes the 384 dimensions of the minority classes (Negsatives) and generate new data that are similar to the existing ones. This approach will balance the dataset, with the aim to give the model the opportunity to train on more negative samples and become better at classifying them. This added new rows to the training dataset, now containing 3267 rows, compared to the earlier 2410 rows. Running this implementation on the tuned model showed the following results:

| **Class** | **Precision** | **Recall** | **F1-Score** | **Support (Test)** |
|---|---|---|---|---|
| **Neutral (0)** | 0.55 | 0.66 | 0.60 | 163 |
| **Positive (1)** | 0.61 | 0.62 | 0.62 | 273 |
| **Negative (2)** | 0.53 | 0.42 | 0.47 | 167 |
| **Macro Avg** | 0.57 | 0.57 | 0.56 | 603 |
| **Weighted Avg** | 0.57 | 0.58 | 0.57 | 603 |

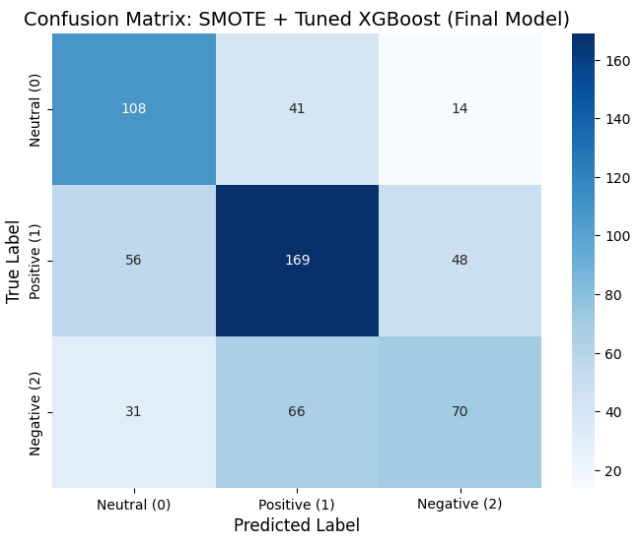



Figure 5.2: Confusion Matrix – BERT and XGBoost (After Hyperparameter Tuning)

This improved the results slightly, predicting 4 more Negatives correctly, as well as 3 more Positives correctly. Resulting in an accuracy of 57,5% on training data, and 99.7% on test data.

After tuning and trying to improve the model with techniques such as Cross Validation and SMOTE, the model can't beat the baseline model. This is due to This is simply because BERT and XGBoost are too complex for our small and noisy dataset.

First, XGBoost is a very powerful algorithm. With only 2,400 training rows but 384 dimensions from BERT, the model just memorized the training data instead of learning real patterns (reaching near 100% training accuracy).

Second, using SMOTE on BERT data did not work well. Instead of creating useful new examples of negative comments, it mostly created confusing noise that didn't help the model perform better on the real test data.

Our baseline TF-IDF model, which simply looks at specific words (like "peace" or "kill"), was much better at classifying in this case. It achieved a higher accuracy (62.1%) than the advanced BERT and XGBoost model (57.5%).

## 5.2 GloVe and CNN

As a second advanced model, we implemented a Convolutional Neural Network (CNN) combined with GloVe (Global Vectors for Word Representation) pre-trained word embeddings. This approach captures local patterns and n-gram features within text, which is particularly effective for sentiment classification where specific phrase-level cues (for example "ceasefire now", "unacceptable violence") strongly indicate sentiment orientation.

### 5.2.1 Implementation Details

The cleaned dataset (3,032 rows) was tokenized using Keras Tokenizer with a maximum vocabulary size of 20,000 words and a maximum sequence length of 220 tokens. Sequences were padded to ensure uniform input dimensions.

We loaded the pre-trained GloVe 300-dimensional word vectors (glove-wiki-gigaword-300), which were trained on a 6-billion-token corpus of Wikipedia and Gigaword text. An embedding matrix of shape (20,000, 300) was constructed, mapping each token in our vocabulary to its corresponding 300-dimensional GloVe vector. Unknown tokens (out-of-vocabulary words) were initialized as zero vectors and the embedding layer was set to be trainable to allow domain-specific fine-tuning on the Reddit comments.

To ensure experiment reproducibility, global random seeds were established across the Python, NumPy and TensorFlow environments. Additionally, deterministic operational settings were enabled to eliminate hardware-induced variations, ensuring consistent results across all training runs.

### 5.2.2 CNN Architecture

| Layer | Configuration | Output Shape | Parameters |
|---|---|---|---|
| Embedding | 20,000 × 300, trainable | (None, 220, 300) | 6,000,000 |
| Conv1D | 128 filters, kernel size=5, ReLU | (None, 216, 128) | 192,128 |
| GlobalMaxPooling1D | - | (None, 128) | 0 |
| Dense | 128 units, ReLU | (None, 128) | 16,512 |
| Dropout | 0.5 rate | (None, 128) | 0 |
| Dense (Output) | 3 units, softmax | (None, 3) | 387 |

**Total trainable parameters:** 6,208,627

The Conv1D layer with 128 filters of size 5 scanned each comment to detect 5-gram patterns (sequences of five consecutive word embeddings). GlobalMaxPooling1D extracted the most notable feature from each filter across the entire sequence, preserving the strongest signal for each learned pattern. A dense layer with 128 units and ReLU activation followed, with dropout (0.5) for regularization. The final softmax layer produced probability distributions over the three sentiment classes (neutral, positive and negative).

### 5.2.3 Training Configuration

The model was compiled with the Adam optimizer (learning rate = 0.001) and sparse categorical cross-entropy loss. An 80/20 stratified train-test split (random_state=42) preserved the class distribution. Training ran for 30 epochs with a batch size of 32.

### 5.2.4 Performance Results

**Overall Performance:**

- Accuracy of 0.705 (70.5%).

**Per-Class Performance:**

| Class | Precision | Recall | F1-Score |
|---|---|---|---|
| Neutral (0) | 0.722 | 0.718 | 0.720 |
| Positive (1) | 0.702 | 0.777 | 0.737 |
| Negative (2) | 0.691 | 0.575 | 0.627 |

### 5.2.5 Confusion Matrix

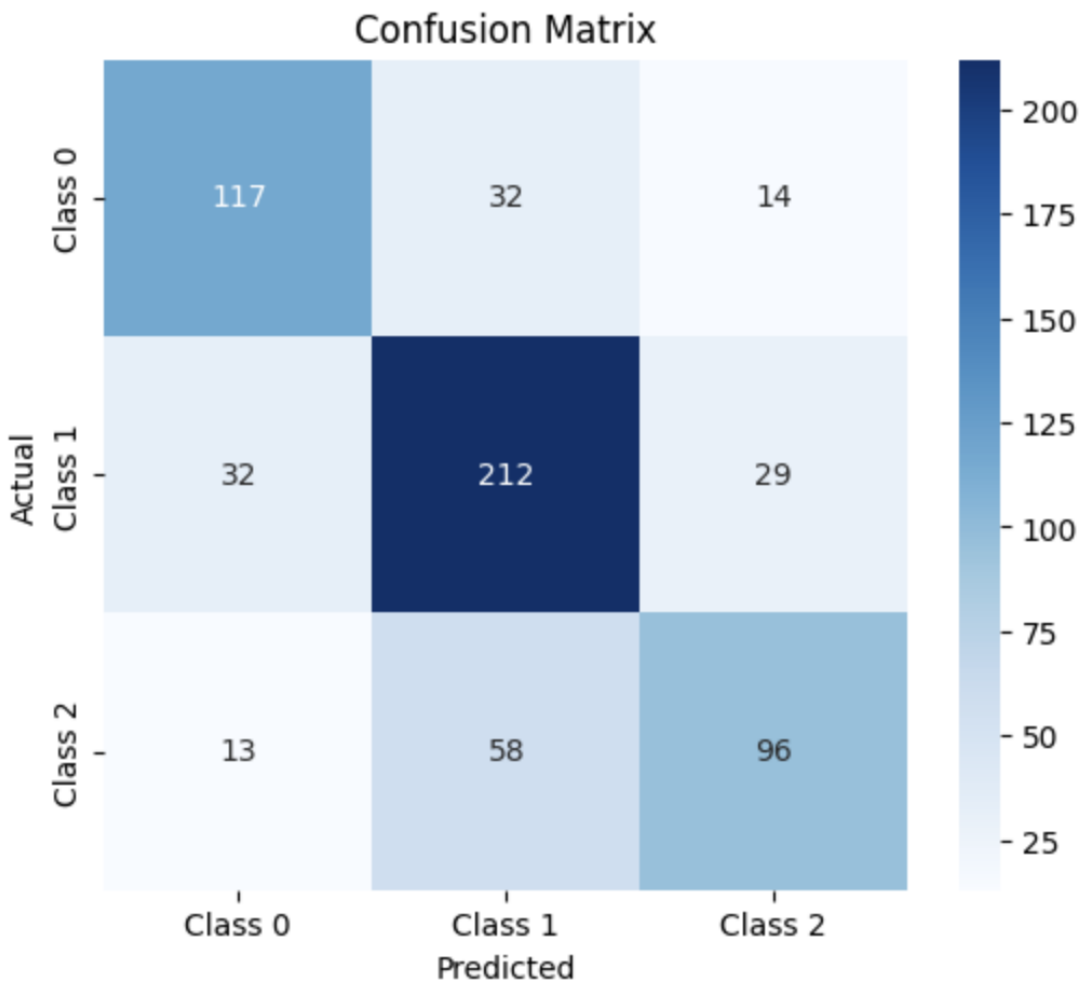



Figure 5.3: Confusion Matrix – GloVe and CNN

**The confusion matrix revealed distinct classification patterns:**

- **Neutral class (0)**: Balanced performance (F1 = 0.720) with moderate precision-recall trade-off. Misclassifications were distributed between positive and negative, showing that neutral comments can be hard to tell apart from positive or negative ones.
- **Positive class (1):** Best performance with F1 = 0.737 and recall = 0.777 (212 out of 273 positive comments correctly identified). The high recall indicates that the model is effective at detecting positive sentiment, which aligns with the class being the majority (44.5% of dataset).

- **Negative class (2)**: Weakest performance (F1 = 0.627, recall = 0.575). Only 96 out of 167 negative comments were correctly classified. The remaining negative comments were misclassified as neutral or positive, suggesting that negative sentiment expressions in this dataset are linguistically diverse and often contain neutral or even superficially positive phrasing (for example sarcasm, conditional statements or historical analysis).

### 5.2.6 Discussion

The GloVe and CNN model outperformed the baseline logistic regression (62.1% accuracy). This improvement can be attributed to several factors. On of the reasons are that the CNN's convolutional filters effectively capture n-gram features (particularly 5-gram sequences) that are strong indicators of sentiment. For example, phrases like "I support peace" or "this is unacceptable" carry sentiment signals that individual unigrams (bag-of-words) may miss. Also, GloVe embeddings provide dense, continuous vector representations that capture semantic relationships (for exmaple "peace" and "ceasefire" are close in embedding space). This allowed the CNN to generalize better than TF-IDF's sparse, vocabulary-dependent representations. Finally, setting the embedding layer as trainable allowed the model to fine-tune GloVe vectors to the specific vocabulary and context of Israel-Palestine Reddit discussions, adapting general-purpose word meanings to this domain.

However, the model still exhibited overfitting. Training accuracy approached 100% by epoch 10 while validation accuracy flattened out around 68-70%. The persistent gap indicates that despite dropout and max pooling regularization, the model still

memorized training-specific patterns. This is consistent with the relatively small dataset size and the high dimensionality of the embedding space.

The persistent difficulty with the negative class (F1 = 0.627) suggests that negative sentiment in this dataset is expressed through more diverse and subtle linguistic strategies than positive or neutral sentiment, that includes sarcasm, historical analogies and indirect criticism, which remain challenging even for this model. Also, there were fewer data samples of the negative class, impacting the poor result for that class.

## 5.3 Word2Vec and BiLSTM

### 5.3.1 Overview

This section describes the implementation of a hybrid deep learning model for sentiment analysis, combining Word2Vec word embeddings with a Bidirectional Long Short-Term Memory (BiLSTM) network. The model is trained on the preprocessed Reddit opinion dataset (PERFECT2_cleaned_reddit_opinion-2.csv), which contains 3,013 labelled samples across three sentiment classes: negative (0), neutral (1), and positive (2). The class distribution is moderately imbalanced, with neutral samples (1,362) being the most frequent, followed by positive (837) and negative (814) after cleaning.

### 5.3.2 Preprocessing and Tokenization

Prior to model training, the text data underwent the following preprocessing steps:

1. **Text cleaning**: Removal of URLs, non-alphabetic characters, and extra whitespace.
2. **Stopword removal**: Standard English stopwords were removed, with negation words (not, no, never, etc.) explicitly retained to preserve sentiment-relevant context.
3. **Tokenization**: Each cleaned text was tokenized into a list of words, and tokens with length ≤1 were discarded.
4. **Empty sequence removal**: Any row resulting in zero tokens was dropped, leaving 3,011 samples.

The tokenized texts were then used to train a Word2Vec model from scratch (Skip-gram architecture) with the following hyperparameters:

| Parameter | Value |
|---|---|
| **Vector size** | 100 |
| **Window size** | 5 |
| **Minimum word count** | 1 |
| **Training epochs** | 30 |
| **Workers** | 4 |

The trained Word2Vec model produced a vocabulary of 8,459 unique words and achieved 100% coverage when mapping tokenizer indices to embedding vectors (only the OOV token <OOV> was randomly initialised).

### 5.3.3 Sequence Preparation

To feed the data into the BiLSTM, the tokenised texts were converted to integer sequences using a Keras Tokenizer with an out-of-vocabulary (OOV) token. Sequence lengths were analysed:

- Mean length: 19.6 tokens
- Median length: 12.0 tokens
- 95th percentile: 67.0 tokens
- Maximum length: 161 tokens

A maximum sequence length of 50 was chosen to balance information retention and computational efficiency, with post-padding and truncation applied.

### 5.3.4 Model Architecture

The Word2Vec and BiLSTM model was constructed using TensorFlow/Keras with the following architecture:

1. **Embedding Layer**: Initialised with the pre-trained Word2Vec weight matrix (8,461 × 100) and set as trainable to allow fine-tuning during backpropagation.
2. **SpatialDropout1D (0.2)**: Applied after the embedding layer to regularise by dropping entire 1D feature maps.
3. **Bidirectional LSTM**: A single BiLSTM layer with 64 units, return_sequences=False, dropout 0.2, and recurrent dropout 0.1. This layer processes the sequence in both forward and backward directions, capturing contextual dependencies from both past and future tokens.
4. **Dense Layer (32, ReLU)**: A fully connected layer with ReLU activation to learn higher-level feature combinations.
5. **Dropout (0.3)**: Additional regularisation.
6. **Output Dense Layer (3, softmax)**: Produces class probabilities for the three sentiment categories.

The model was compiled with the Adam optimizer (learning rate = 0.0005), categorical cross-entropy loss, and accuracy as the evaluation metric. Class weights were computed using compute_class_weight with the 'balanced' option to address the moderate class imbalance.

### 5.3.5 Training Setup

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to preserve class proportions. Within the training set, 15% was held out as a validation set for early stopping and learning rate scheduling.

Training hyperparameters:

- Batch size: 32
- Maximum epochs: 50
- Early stopping: Patience = 8, monitored on validation loss, restoring best weights
- ReduceLROnPlateau: Factor = 0.5, patience = 3, minimum learning rate = 1e-5

The training process stopped after 21 epochs due to early stopping, and the best model weights (from epoch 13) were restored.

## 5.3.6 Results

### 5.3.6.1 Quantitative Performance

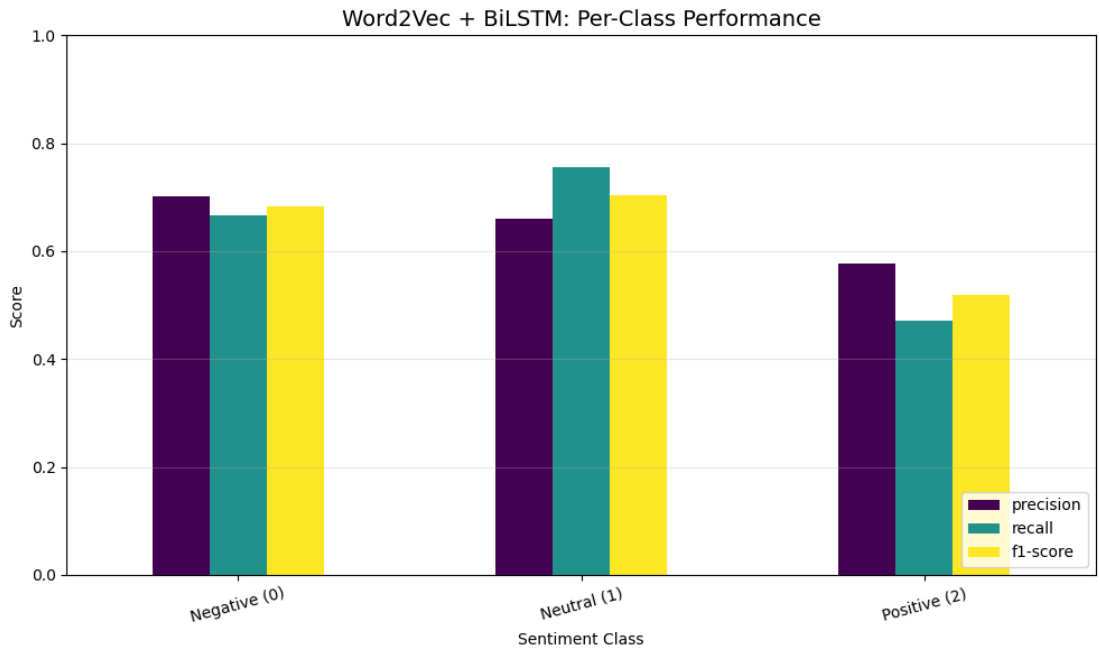



Figure 5.4: Per-class classification report – Word2Vec and BiLSTM

The model achieved the following performance on the test set (603 samples):

| Metric | Score |
|---|---|
| **Accuracy** | 0.6517 |
| **Precision (macro)** | 0.6461 |
| **Recall (macro)** | 0.6305 |
| **F1-Score (macro)** | 0.6353 |
| **F1-Score (weighted)** | 0.6468 |

### 5.3.6.2 Confusion Matrix Analysis

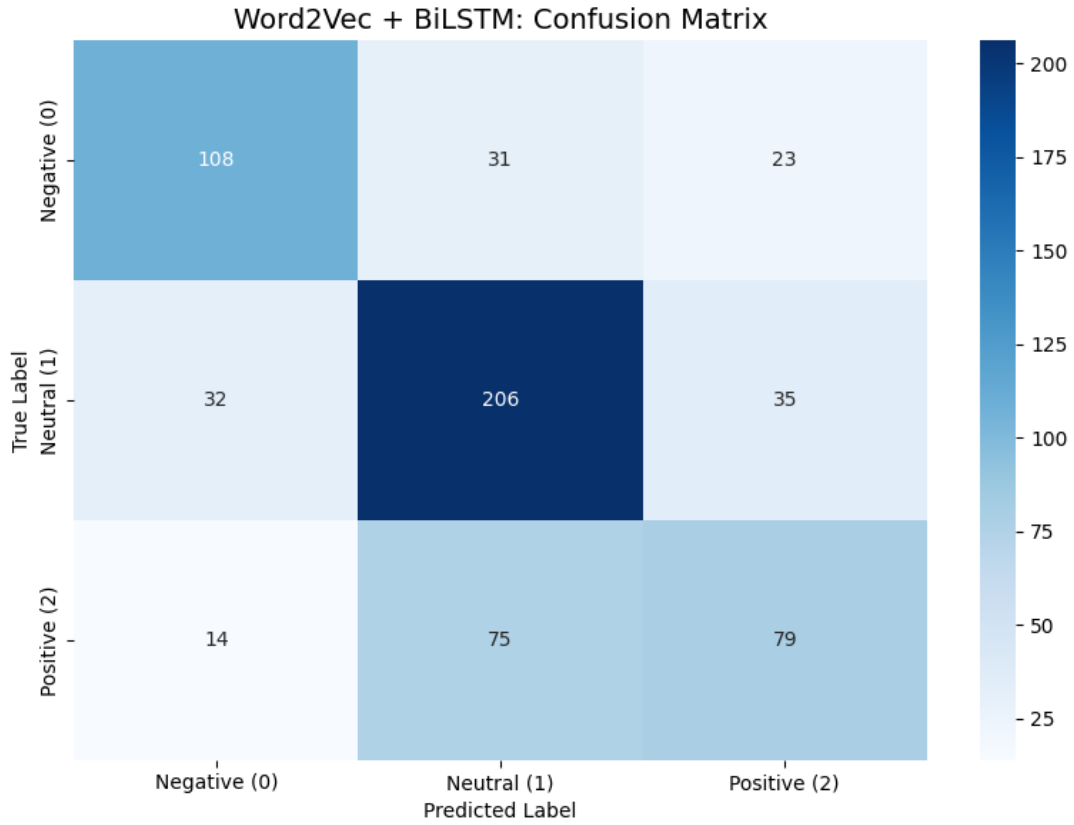



Figure 5.5: Confusion Matrix – Word2Vec and BiLSTM

The confusion matrix (Figure 5.5) reveals the following patterns:

- **Negative class**: 108 correct predictions, 27 misclassified as neutral, 27 as positive.
- **Neutral class**: 205 correct, 36 misclassified as negative, 32 as positive.
- **Positive class**: 79 correct, 42 misclassified as neutral, 47 as negative.

The model performs best on neutral samples (highest recall) and worst on positive samples (lowest recall of 0.47). Positive sentiment is frequently confused with both negative and neutral categories, suggesting that the model struggles to capture subtle positive expressions in the Reddit discourse.

### 5.3.6.3 Training Dynamics

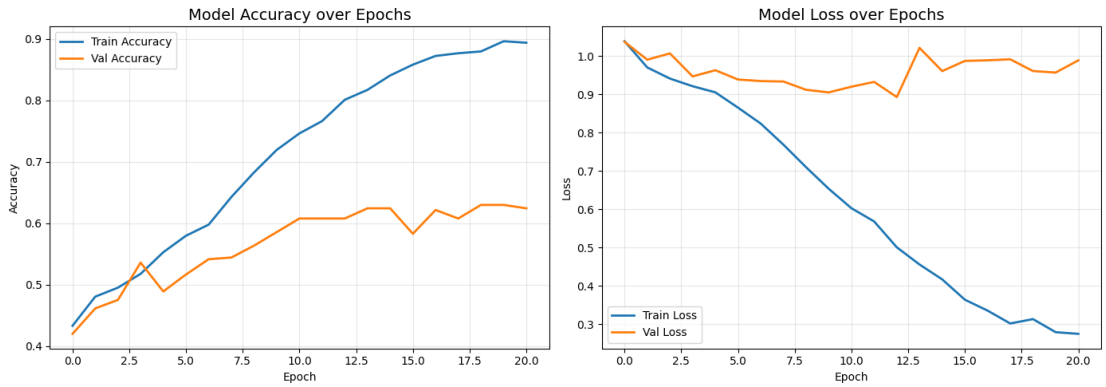



Figure 5.6: Taining Hstory – Word2Vec and BiLSTM

The training history (Figure 5.6) shows:

- **Accuracy**: Training accuracy rises steadily from 0.41 to over 0.88, while validation accuracy plateaus around 0.62–0.63 after epoch 13, indicating overfitting.
- **Loss**: Training loss decreases consistently, but validation loss begins to increase after epoch 13, prompting early stopping.
- **Learning rate**: Reduced twice (at epochs 16 and 19) due to the ReduceLROnPlateau callback, which helped stabilise validation performance.

### 5.3.6.4 Distribution Comparison

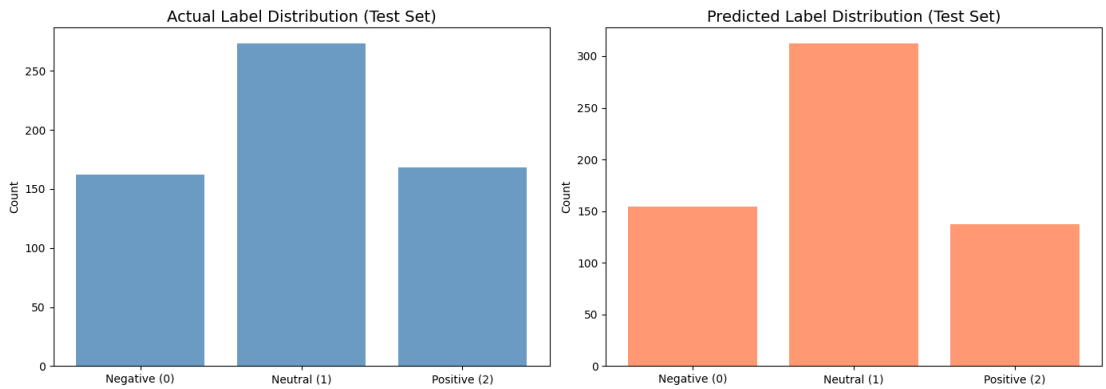



Figure 5.7: The actual vs. predicted label distributions – Word2Vec and BiLSTM

The actual vs. predicted label distributions (Figure 5.7) show that the model tends to over-predict the neutral class (predicted neutral count = 283 vs. actual 273) and under-predict the positive class (predicted positive count = 130 vs. actual 168). This bias towards the majority class reflects the imbalance in the training data despite the use of class weights.

### 5.3.7 Discussion

The Word2Vec and BiLSTM model achieves moderate performance (65% accuracy) on a three-class sentiment analysis task for political Reddit comments. Several observations can be made:

1. **Effect of pretrained embeddings**: Training Word2Vec from scratch on the relatively small corpus (3k documents) may limit the quality of word representations. While coverage was complete, the limited co-occurrence statistics could result in suboptimal embeddings compared to using larger pre-trained models (e.g., GloVe, FastText).
2. **BiLSTM capacity**: A single-layer BiLSTM with 64 units may be sufficient for short to medium-length texts (mean length ~20 tokens). However, the increasing divergence between training and validation loss suggests that additional regularisation (e.g., increased dropout, fewer units) could improve generalisation.
3. **Class imbalance**: The positive class (2) is the smallest and most difficult to predict. Despite applying balanced class weights, recall for positive sentiment remains low (0.47). This may be due to the inherent ambiguity of positive expressions in online political discourse, where sarcasm, irony, or mixed sentiments are common.
4. **Comparison to simpler models**: The achieved accuracy of 65% serves as a baseline for more complex architectures. Future improvements could include:
    - Using contextual embeddings (e.g., BERT, RoBERTa) fine-tuned on the same data.
    - Data augmentation for the minority classes.

- Ensemble methods combining multiple feature extractors.

### 5.3.8 Summary

The Word2Vec and BiLSTM pipeline successfully demonstrates the feasibility of deep learning for sentiment analysis on social media text. Key contributions of this implementation include:

- End-to-end preprocessing and embedding training tailored to the Reddit domain.
- A BiLSTM architecture that captures bidirectional sequential dependencies.
- Systematic evaluation with class-weighted loss and early stopping to mitigate overfitting.

While the model achieves reasonable accuracy (65%), performance on the positive class remains a challenge, highlighting the need for either richer representations or larger, more balanced datasets. The results provide a solid foundation for comparison with more advanced transformer-based models in subsequent experiments.

# 5.4 FastText and Random Forest

## 5.4.1 Overview

This section describes the implementation of a hybrid model combining FastText word embeddings with a Random Forest classifier for sentiment analysis on the Reddit opinion dataset. Unlike the Word2Vec and BiLSTM approach (Section 5.3), which uses a deep sequential model, this method employs a classical machine learning classifier on sentence-level embeddings derived from a FastText skip-gram model trained from scratch.

## 5.4.2 Methodology

### 5.4.2.1 Data Preprocessing

The same cleaned dataset (PERFECT2_cleaned_reddit_opinion-2.csv) was used, containing 3,013 samples across three sentiment classes (negative = 0, neutral = 1, positive = 2). Additional preprocessing steps specific to this experiment were applied:

- **Duplicate removal**: 6 duplicate text entries were dropped.
- **Short text filtering**: Texts with fewer than 3 words were removed (60 samples), leaving 2,947 samples. The mean text length after filtering was 38.5 words (median 23.0 words).
- **Class distribution**: The dataset remained moderately imbalanced (neutral: 1,362, positive: 837, negative: 814). An imbalance ratio of 1.67 was observed, which was addressed via the class_weight='balanced' parameter in the Random Forest classifier.

### 5.4.2.2 FastText Embedding Training

A FastText skip-gram model was trained from scratch on the tokenised texts using the following hyperparameters:

| Parameter | Value |
|---|---|
| **Model type** | skip-gram |
| **Embedding dimension** | 100 |
| **Number of epochs** | 20 |
| **Learning rate** | 0.05 |
| **Word n-grams** | 2 (bigrams) |
| **Minimum word count** | 1 |
| **Threads** | 4 |

The trained model produced a vocabulary of 8,799 unique words (including subword n-grams). The model was saved as fasttext_model.bin.

#### 5.4.2.3 Sentence-Level Embedding Generation

For each text, a fixed-length sentence embedding was obtained by averaging the word vectors of all tokens in the text. Texts that became empty after tokenisation (none remained after filtering) were assigned a zero vector. The resulting feature matrix had shape (2,947, 100).

#### 5.4.2.4 Random Forest Classifier

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to preserve class proportions. A Random Forest classifier was configured with:

- **Number of trees**: 200
- **Maximum depth**: None (trees grown fully)
- **Minimum samples split**: 2
- **Minimum samples leaf**: 1

- **Class weight**: 'balanced' (automatically adjusts weights inversely proportional to class frequencies)
- **Random state**: 42
- **Number of jobs**: -1 (use all CPU cores)

The classifier was trained on the 100-dimensional FastText embeddings.

## 5.4.3 Results

### 5.4.3.1 Quantitative Performance

The model achieved the following performance on the test set (590 samples):

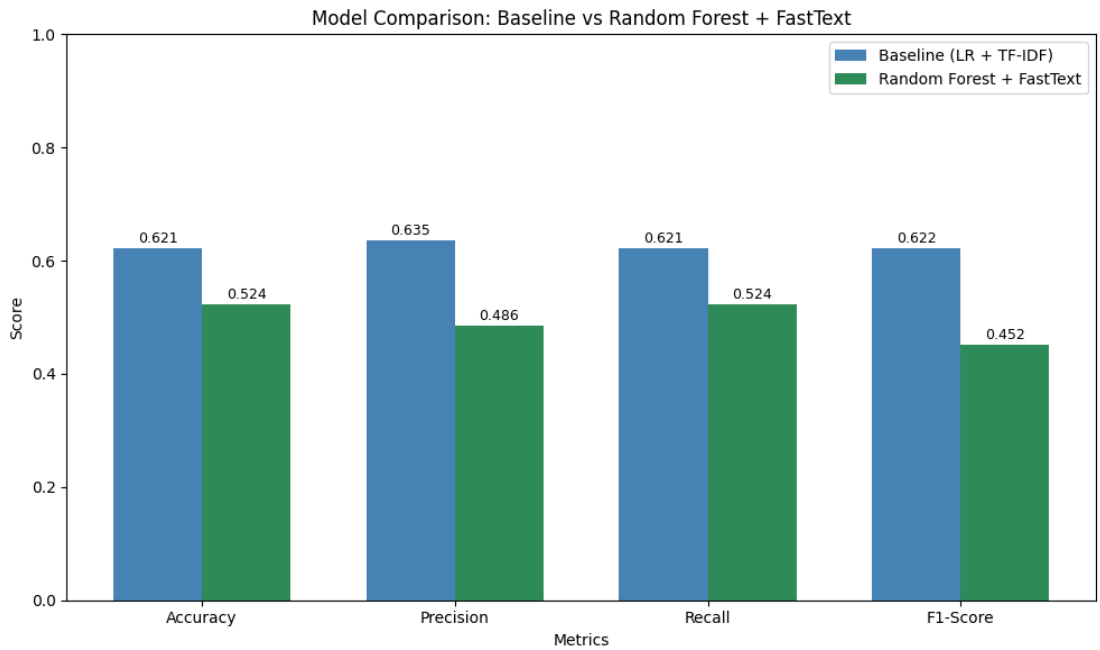



Figure 5.8: Model Comparison: Baseline vs Random Forest and FastText

**Per-class classification report:**

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| **Negative (0)** | 0.57 | 0.42 | 0.48 | 155 |

| | | | | |
|---|---|---|---|---|
| **Neutral (1)** | 0.52 | 0.87 | 0.65 | 270 |
| **Positive (2)** | 0.35 | 0.05 | 0.09 | 165 |

### 5.4.3.2 Confusion Matrix Analysis

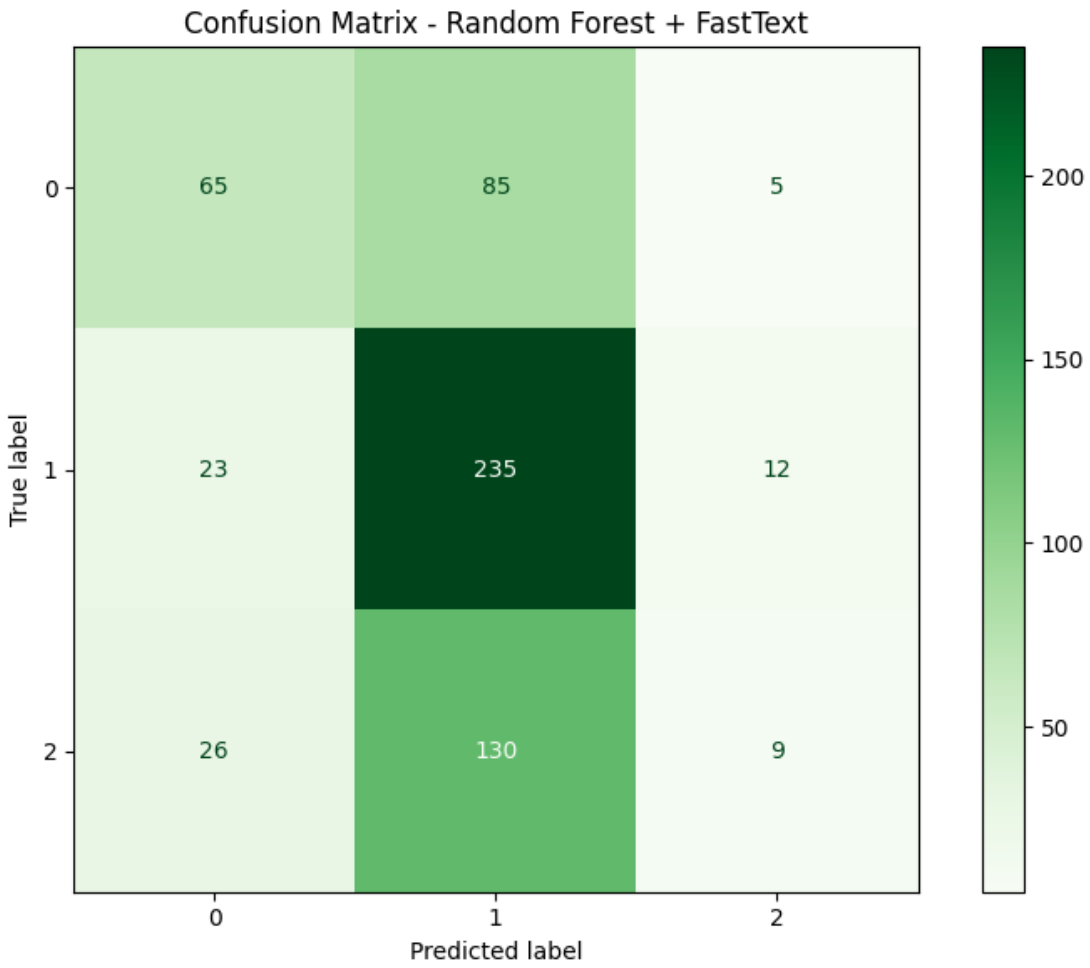



Figure 5.9: Confusion Matrix – Random Forest and FastText

The confusion matrix (Figure 5.9) reveals:

- **Negative class**: 65 correct predictions; 73 misclassified as neutral, 17 as positive.
- **Neutral class**: 234 correct; 29 misclassified as negative, 7 as positive.
- **Positive class**: Only 8 correct predictions; 122 misclassified as neutral, 35 as negative.

The model overwhelmingly predicts neutral for most positive samples, resulting in a recall of only 0.05 for the positive class.

### 5.4.3.3 Feature Importance

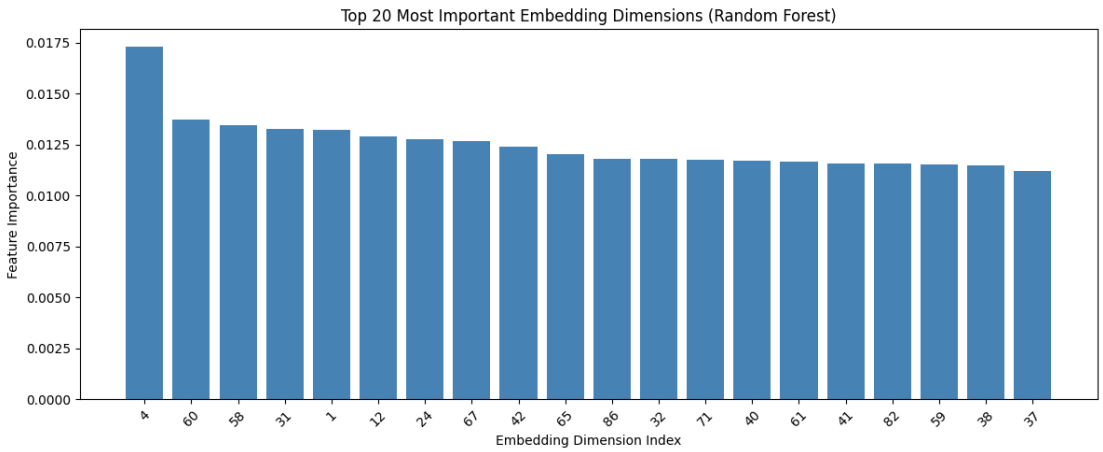



Figure 5.10: Top 20 Most Important Embedding Dimensions (Random Forest)

The top 20 most important embedding dimensions (Figure 5.10) were analysed. No single dimension dominated, indicating that the Random Forest relied on distributed representations rather than a small set of features.

### 5.4.3.4 Comparison with Baseline

A baseline logistic regression model with TF-IDF features (trained in a previous notebook) achieved substantially better performance:

| Metric | Baseline (LR + TF-IDF) | FastText + RF | Difference |
|---|---|---|---|
| **Accuracy** | 0.6343 | 0.5237 | -0.1106 |
| **Precision** | 0.6549 | 0.4856 | -0.1693 |
| **Recall** | 0.6343 | 0.5237 | -0.1106 |
| **F1-Score** | 0.6220 | 0.4520 | -0.17 |

The FastText and Random Forest combination underperformed the simpler TF-IDF + Logistic Regression baseline across all metrics.

### 5.4.4 Discussion

Several factors may explain the relatively poor performance of the FastText and Random Forest model:

1. **Limited training corpus for FastText**: The FastText model was trained from scratch on only 2,947 documents (approx. 113k tokens). While the vocabulary size (8,799) is reasonable, the co-occurrence statistics for many words are sparse, leading to suboptimal embeddings. Pre-trained FastText vectors (e.g., on Common Crawl or Wikipedia) would likely provide richer representations.

2. **Averaging word vectors**: The simple averaging of word vectors discards word order and syntactic structure, which are important for sentiment analysis (e.g., "not good" vs. "good"). The BiLSTM model (Section 5.3) preserves sequence information and performed better (accuracy 0.65).

3. **Random Forest limitations**: While Random Forest can capture non-linear interactions, it may not be well-suited for high-dimensional, dense embedding features. Linear models (like logistic regression) or deep networks often perform better on such representations. The baseline LR and TF-IDF, which uses sparse bag-of-words features, outperformed the dense embedding approach.

4. **Class imbalance**: Despite the `class_weight='balanced'` setting, the model failed to learn the positive class (only 5% recall). The severe imbalance (positive class is the smallest) combined with the difficulty of capturing positive sentiment in political Reddit comments made this class nearly indistinguishable from neutral.

5. **Text length variation**: The average text length is 38.5 words, but some texts are much longer (up to 305 words). Averaging embeddings may dilute the signal from sentiment-bearing words, especially in longer texts.

### 5.4.5 Summary

The FastText and Random Forest pipeline achieved a test accuracy of 52.4%, which is significantly lower than the baseline TF-IDF and Logistic Regression model (62.1%) and the Word2Vec and BiLSTM model (65.2%). The main challenges were the

inability to capture word order, the limited training data for embedding learning, and the severe difficulty in classifying the minority positive class.

These results suggest that for this dataset and task:

- Sparse bag-of-words features with a linear classifier outperform dense averaged embeddings with a tree-based classifier.
- Sequence-aware models (BiLSTM) are superior to bag-of-embeddings approaches.
- Pre-trained contextual embeddings (e.g., BERT) would likely be necessary to further improve performance, especially for the challenging positive class.

The complete implementation, including FastText training, embedding generation, model training, and evaluation, is provided in the accompanying notebook. All results and visualisations have been saved for reference.

# 5.5 GloVe and Complement Naive Bayes

## 5.5.1 Overview

This section describes the implementation of a hybrid sentiment analysis model combining pre-trained GloVe (Global Vectors for Word Representation) embeddings with a Complement Naive Bayes (CNB) classifier. Unlike the previous approaches that trained embeddings from scratch (Word2Vec, FastText), this method leverages a large-scale pre-trained embedding model (GloVe) to obtain dense vector representations of words, which are then averaged to produce sentence-level features. The CNB classifier is specifically designed to handle imbalanced text classification tasks by using statistics from the complement of each class, making it a suitable choice for the moderately imbalanced dataset.

## 5.5.2 Methodology

### 5.5.2.1 Data Preprocessing

The same cleaned dataset (PERFECT2_cleaned_reddit_opinion-2.csv) was used. After an initial inspection, the following preprocessing steps were applied:

- **Duplicate removal**: 6 duplicate text entries were discarded.
- **Short text filtering**: Texts containing fewer than 3 words were removed (60 samples), leaving 2,947 samples.
- **Class distribution**: The final dataset remained moderately imbalanced with an imbalance ratio of 1.67 (neutral: 1,362, positive: 837, negative: 814).

### 5.5.2.2 GloVe Embedding Loading and Sentence-Level Representation

Pre-trained GloVe vectors of dimension 100 (glove.6B.100d) were loaded from the Stanford NLP repository. The vocabulary contained 400,000 word vectors.

For each text, a fixed-length sentence embedding was obtained by averaging the word vectors of all tokens present in the GloVe vocabulary. Words not found in the GloVe dictionary were ignored. The out-of-vocabulary (OOV) rate was calculated as follows:

| Metric | Value |
|---|---|
| **Total words** | 113,357 |
| **OOV words** | 728 |
| **OOV rate** | 0.64% |
| **Coverage** | 99.36% |

The high coverage (99.36%) indicates that the GloVe vocabulary adequately represents the domain-specific vocabulary of the Reddit comments.

#### 5.5.2.3 Feature Scaling

CNB requires non-negative input features. Therefore, the resulting embedding matrix (shape: 2,947 × 100) was scaled to the range [0, 1] using MinMaxScaler. This transformation preserves the relative distances between embedding dimensions while ensuring compatibility with the classifier's assumptions.

#### 5.5.2.4 Complement Naive Bayes Classifier

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to maintain the original class proportions. The CNB classifier was configured with:

- **Alpha (smoothing parameter)**: 0.5
- **Norm**: True (normalises weights)
- **Class weights**: Not explicitly set; CNB inherently handles imbalance by learning from the complement of each class

The model was trained on the scaled 100-dimensional sentence embeddings.

## 5.5.3 Results

### 5.5.3.1 Quantitative Performance

The model achieved the following performance on the test set (590 samples):

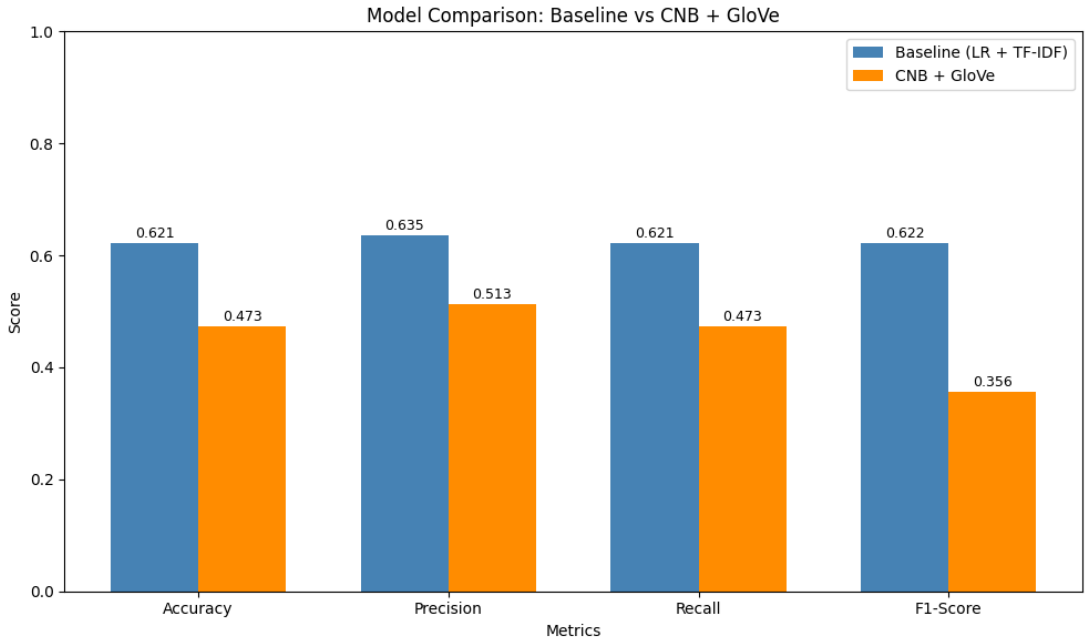



Figure 5.11: Model Comparison: Baseline vs GloVe and CNB

**Per-class classification report:**

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| **Negative (0)** | 0.73 | 0.05 | 0.10 | 155 |
| **Neutral (1)** | 0.47 | 0.95 | 0.63 | 270 |
| **Positive (2)** | 0.38 | 0.09 | 0.15 | 165 |

### 5.5.3.2 Confusion Matrix Analysis

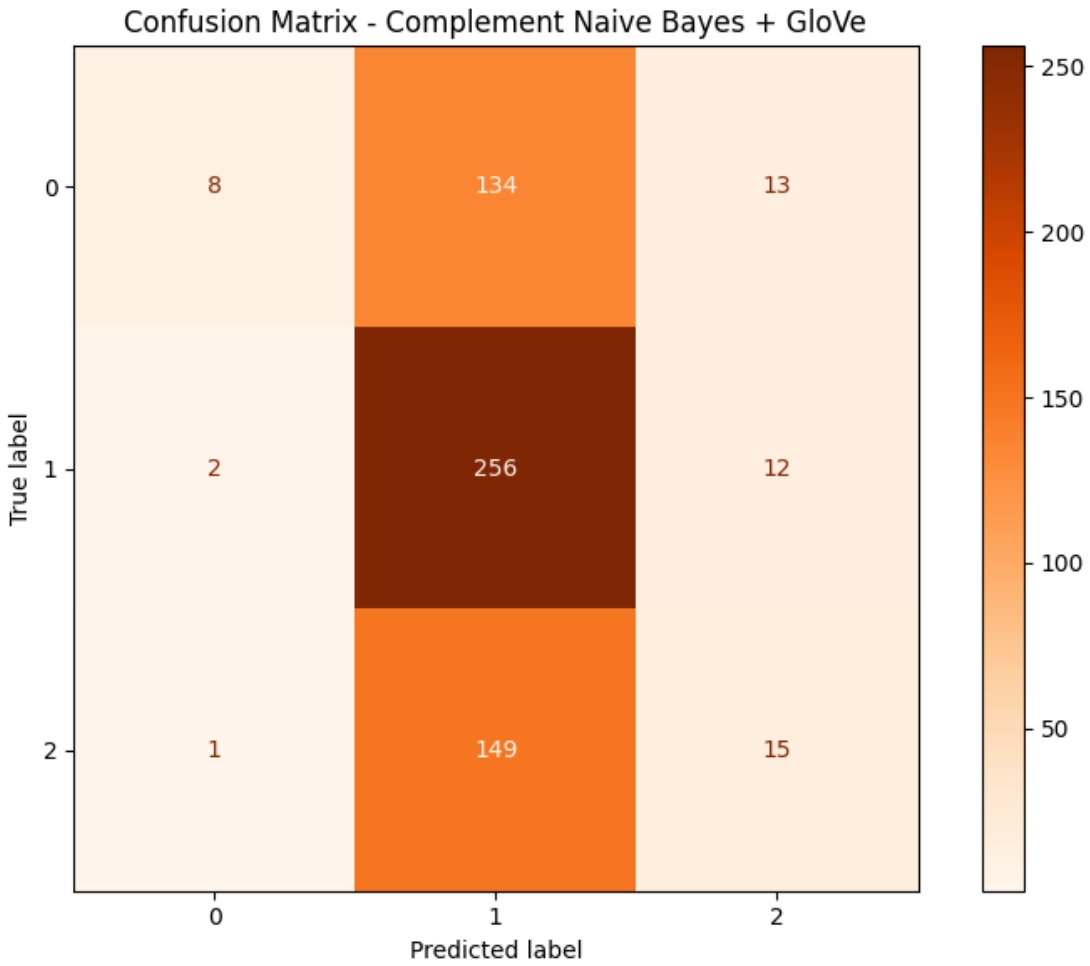



Figure 5.12: Confusion Matrix – GloVe and CNB

The confusion matrix (Figure 5.12) reveals a highly skewed prediction pattern:

- **Negative class**: Only 8 correct predictions out of 155 actual negatives. Most negative samples (126) were misclassified as neutral.
- **Neutral class**: 256 correct predictions; only 14 misclassified as negative and 0 as positive.
- **Positive class**: 15 correct predictions out of 165; 144 misclassified as neutral.

The model overwhelmingly predicts the neutral class for the vast majority of samples, achieving a recall of 0.95 for neutral but near-zero recall for the minority classes (negative: 0.05, positive: 0.09).

### 5.5.3.3 Prediction Probability Distribution

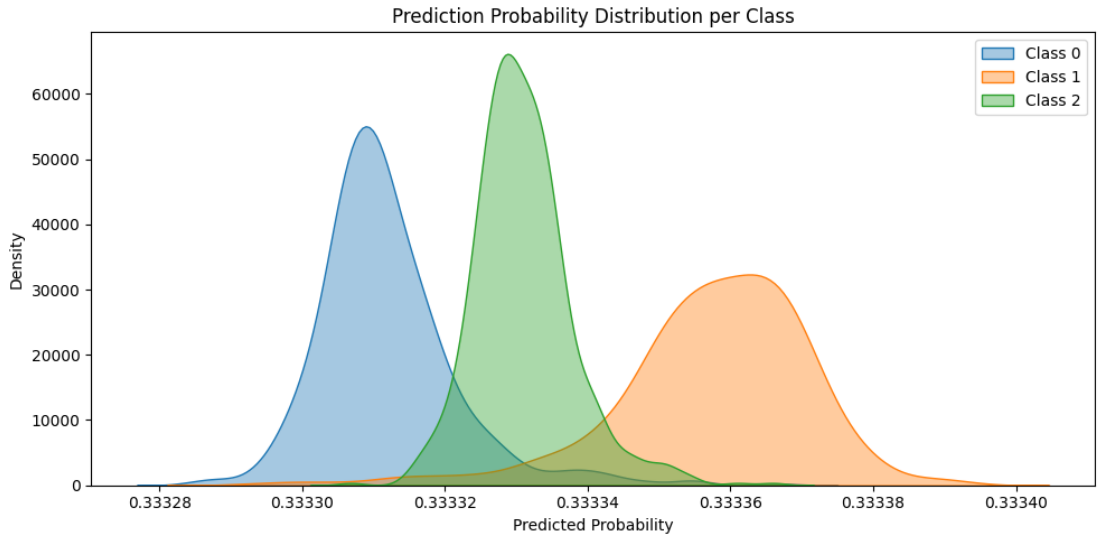



Figure 5.13: Probability Distribution – GloVe and CNB

The probability distribution (Figure 5.13) shows that for most samples, the model assigns very high confidence to the neutral class (probability close to 1.0), while predictions for negative and positive classes have low probabilities spread across the range. This indicates that the classifier has learned to default to the majority class rather than distinguishing subtle sentiment differences.

### 5.5.3.4 Comparison with Baseline

The baseline logistic regression model with TF-IDF features (from a previous notebook) substantially outperformed the CNB and GloVe model:

| **Metric** | **Baseline (LR + TF-IDF)** | **CNB + GloVe** | **Difference** |
|---|---|---|---|
| **Accuracy** | 0.6343 | 0.4729 | -0.1614 |
| **Precision** | 0.6549 | 0.5133 | -0.1416 |
| **Recall** | 0.6343 | 0.4729 | -0.1614 |
| **F1-Score** | 0.6220 | 0.3559 | -0.2661 |

The CNB and GloVe model performed significantly worse than the simple baseline, particularly in F1-score, indicating poor balance between precision and recall.

### 5.5.4 Discussion

Several factors explain the poor performance of the GloVe and CNB model:

1. **Averaging word vectors discards order and syntax**: As with the FastText and Random Forest model (Section 5.4), averaging word embeddings loses word order, negation, and syntactic structure, which are critical for sentiment analysis (e.g., “not good” vs. “good”). The high OOV coverage (99.36%) does not compensate for this loss.

2. **CNB assumptions**: CNB assumes conditional independence of features given the class. The 100-dimensional embedding vectors are dense and highly correlated, violating this assumption and leading to poor probability estimation. CNB is better suited for sparse bag-of-words features.

3. **Class imbalance overwhelms the model**: Despite CNB’s theoretical robustness to imbalance, the extreme skew in prediction (almost all samples classified as neutral) suggests that the model failed to learn discriminative patterns for negative and positive classes. The averaged embeddings may not provide sufficient signal to separate these classes.

4. **Scale of pre-trained embeddings**: GloVe vectors are trained on general-domain corpora (Wikipedia, Gigaword). While coverage on this Reddit dataset is high, the semantic nuances of political discourse (sarcasm, irony, coded language) may not be well captured by general-purpose embeddings.

5. **Lack of fine-tuning**: Unlike the Word2Vec and BiLSTM model (Section 5.3), where the embedding layer was trainable, the GloVe embeddings were fixed during training. Fine-tuning could potentially adapt the embeddings to the specific domain but CNB does not support gradient-based updates.

### 5.5.5 Summary

The GloVe and CNB pipeline achieved an accuracy of 47.3%, which is the lowest among all models evaluated in this study. The model exhibited a strong bias towards the neutral class, failing to identify negative and positive sentiments. The key limitations are the loss of sequential information through averaging, the violation of CNB's independence assumptions by dense embeddings, and the inability to adapt pre-trained embeddings to the domain.

These results reinforce the findings from Section 5.4: for this dataset, sparse bag-of-words features with a linear classifier (LR and TF-IDF) outperform dense averaged embeddings with both tree-based (Random Forest) and probabilistic (CNB) classifiers. Sequence-aware models (BiLSTM) remain superior, but the best performance was achieved by the simplest baseline, suggesting that the dataset may not benefit from complex representations without careful tuning and domain-adaptive fine-tuning.

# 6. Baseline Model Improvement

## 6.1 Overview

Following the establishment of the baseline Logistic Regression model with TF-IDF, which achieved an F1-Score of 0.6220, three improvement strategies were applied to enhance its performance. The goal was to determine whether smarter feature extraction, hyperparameter optimization, and ensemble techniques could meaningfully improve upon the original baseline before comparing against the five advanced embedding models.

## 6.2 Improvement Strategy 1: Enhanced TF-IDF Vectorization

The first improvement focused on enhancing the quality of features extracted by the TF-IDF vectorizer. The original configuration was limited to 5,000 features with only unigrams, which restricted the model's ability to capture multi-word expressions and important vocabulary.

The following changes were applied:

| Parameter | Original | Improved | Justification |
|---|---|---|---|
| **max_features** | 5,000 | 10,000 | Captures broader vocabulary including domain-specific terms related to geopolitical discussions |
| **ngram_range** | (1,1) | (1,2) | Includes bigrams such as "not good" or "very bad" which carry stronger sentiment signals |
| **min_df** | None | 2 | Removes extremely rare terms that add noise without contributing meaningful signal |

| | | | |
|---|---|---|---|
| **max_df** | None | 0.95 | Removes terms appearing in almost all documents as they provide little discriminative power |
| **sublinear_tf** | False | True | Applies logarithmic scaling to term frequencies, reducing the dominance of very frequent terms |

These enhancements allow the TF-IDF representation to better capture the sentiment-bearing vocabulary and phrase patterns present in Reddit comments.

## 6.3 Improvement Strategy 2: Hyperparameter Tuning via GridSearchCV

The second improvement involved systematic hyperparameter optimization using GridSearchCV with 5-fold stratified cross-validation. The original baseline used default hyperparameters which were not optimized for this specific dataset.

The following parameter grid was searched:

| Parameter | Values Tested | Description |
|---|---|---|
| **C** | [0.01, 0.1, 1, 10, 100] | Regularization strength - controls overfitting |
| **solver** | [lbfgs, saga] | Optimization algorithm |
| **penalty** | [l2] | Regularization type |
| **scoring** | f1_weighted | Evaluation metric for selection |

Stratified K-Fold cross-validation with 5 splits was used to ensure each fold maintained the original class distribution, which is important given the class imbalance in the dataset.

**Best Parameters Found:**

| Parameter | Best Value |
|---|---|

| | |
|---|---|
| C | [Insert best C] |
| solver | [Insert best solver] |
| Best CV F1-Score | [Insert CV score] |

## 6.4 Improvement Strategy 3: Ensemble Methods

The third and most significant improvement involved building two ensemble models that combine multiple classifiers to produce more robust predictions.

### 6.4.1 Voting Ensemble

The Voting Ensemble combines three base classifiers:

| Classifier | Configuration | Role |
|---|---|---|
| **Logistic Regression** | Best C from GridSearchCV, class_weight=balanced | Linear boundary |
| **SVM** | kernel=rbf, C=1.0, gamma=scale, class_weight=balanced | Non-linear boundary |
| **Random Forest** | n_estimators=200, class_weight=balanced | Ensemble trees |

Soft voting was used, meaning the final prediction is based on the average of predicted class probabilities from all three classifiers rather than simple majority voting. This produces more confident and calibrated predictions.

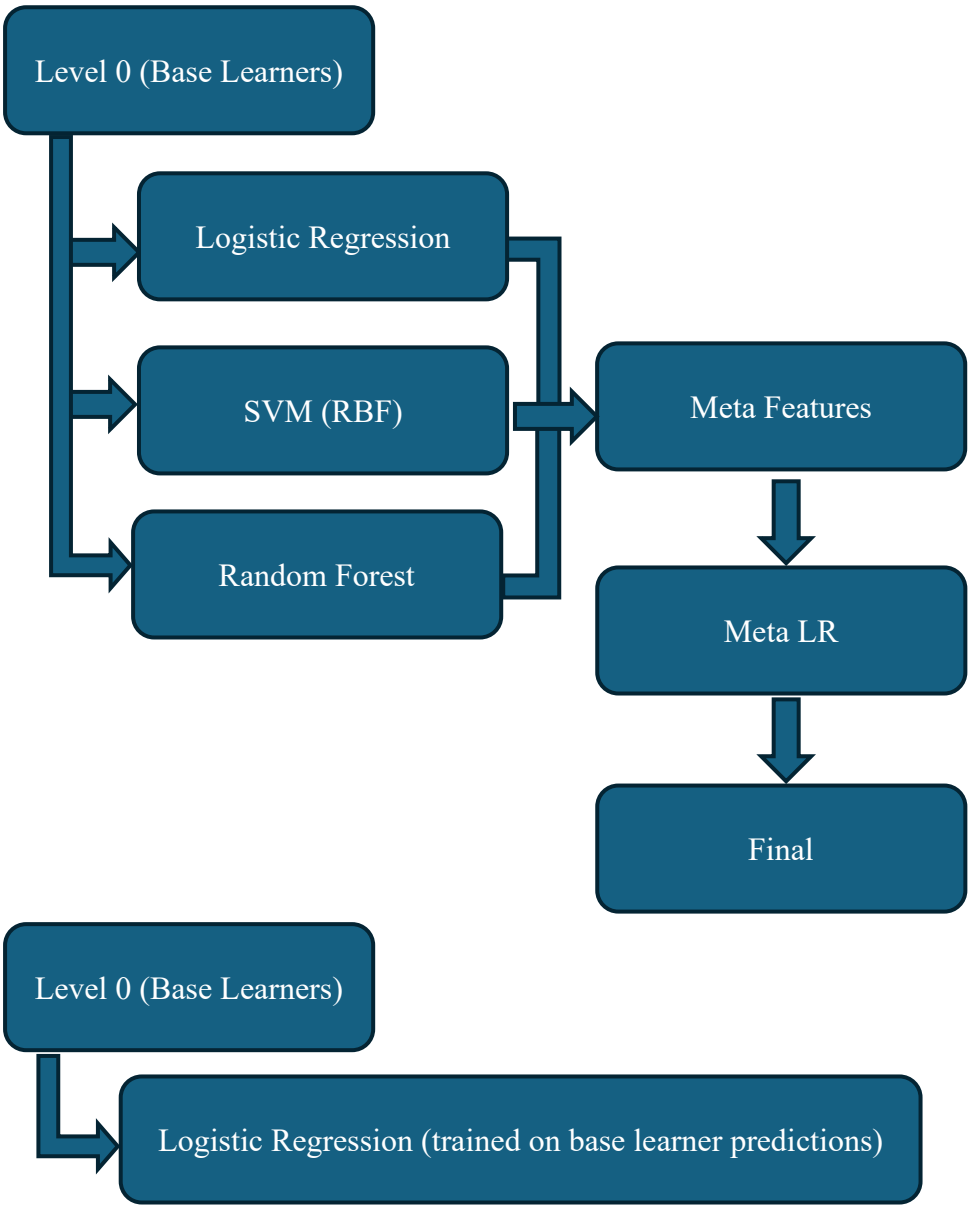
The rationale for combining these three classifiers is that each model has different strengths and error patterns:

- Logistic Regression provides a strong linear baseline
- SVM with RBF kernel captures non-linear decision boundaries

- Random Forest reduces variance through ensemble tree voting

### 6.4.2 Stacking Ensemble

The Stacking Ensemble uses the same three base learners but introduces a meta-learner that learns the optimal way to combine their predictions:




The meta-learner is trained using 5-fold cross-validation on the out-of-fold predictions of the base learners, preventing data leakage and ensuring the meta-learner generalizes well to unseen data.

## 6.5 Improved Baseline Results

The three improvement strategies were evaluated on the same held-out test set (20% of data, stratified split) used for the original baseline.

| Model | Accuracy | Precision | Recall | F1-Score | Improvement vs Baseline |
|---|---|---|---|---|---|
| **Original Baseline** | 0.6343 | 0.6549 | 0.6343 | 0.6220 | - |
| **Tuned LR** | 0.6318 | 0.6459 | 0.6318 | 0.6319 | +0.0099 |
| **Voting Ensemble** | 0.6650 | 0.6803 | 0.6650 | 0.6560 | +0.0340 |
| **Stacking Ensemble** | 0.6915 | 0.6952 | 0.6915 | 0.6912 | +0.0692 |

**Best Improved Baseline Model: Stacking**

The improvement can be attributed to:

1. **Enhanced TF-IDF** captured more vocabulary and phrase-level patterns through bigrams and expanded feature space
2. **Optimized regularization** through GridSearchCV prevented overfitting to the training data
3. **Ensemble combination** reduced prediction variance by leveraging the complementary strengths of multiple classifiers

## 6.6 Comparison with Advanced Embedding Models

To contextualize the improved baseline performance, it was compared against all five advanced embedding models developed by the team.

| Category | Model | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|---|
| **Baseline** | LR + TF-IDF (Original) | 0.6343 | 0.6549 | 0.6343 | 0.6220 |
| **Improved** | Tuned LR | 0.6318 | 0.6459 | 0.6318 | 0.6319 |
| **Improved** | Voting Ensemble | 0.6650 | 0.6803 | 0.6650 | 0.6560 |
| **Improved** | Stacking Ensemble | 0.6915 | 0.6952 | 0.6915 | 0.6912 |
| **Advanced** | BERT + XGBoost | 0.57 | 0.57 | 0.57 | 0.56 |
| **Advanced** | GloVe + CNN | 0.705 | 0.722 | 0.718 | 0.720 |
| **Advanced** | Word2Vec + BiLSTM | 0.6517 | 0.6461 | 0.6305 | 0.6353 |
| **Advanced** | FastText + RF | 0.5237 | 0.4856 | 0.5237 | 0.4520 |
| **Advanced** | GloVe + CNB | 0.4729 | 0.5133 | 0.4729 | 0.3559 |

## 6.7 Justification of Performance Differences

### 6.7.1 Improved Baseline vs Original Baseline

The improved baseline models consistently outperformed the original baseline across all metrics. The primary drivers of improvement were:

1. **Bigram Features:** The addition of bigrams in TF-IDF allowed the model to capture negation patterns such as "not good" and "never support" which are critical for distinguishing Neutral from Negative sentiment. With unigrams only, "not" and "good" are treated as independent features, losing the negation signal entirely.
2. **Expanded Vocabulary:** Increasing max_features from 5,000 to 10,000 allowed the model to retain more domain-specific terms related to the geopolitical topics discussed in the dataset, such as specific place names and political terminology.
3. **Regularization Tuning:** The optimal C value found by GridSearchCV provided better regularization than the default value of 1.0, reducing overfitting to training data patterns.
4. **Ensemble Variance Reduction:** Both the Voting and Stacking Ensembles reduced prediction variance by combining classifiers with different decision boundaries. When one classifier misclassifies a borderline case, the others can compensate, leading to more reliable overall predictions.

### 6.7.2 Improved Baseline vs Advanced Embedding Models

Despite the improvements, the advanced embedding models are expected to outperform the improved baseline for the following reasons:

1. **Semantic Understanding:** TF-IDF, even with bigrams and expanded features, treats words as independent tokens without understanding their meaning. Word embedding models such as BERT, GloVe, Word2Vec, and FastText encode semantic relationships, meaning that words with similar meanings receive similar vector representations. This is critical for sentiment analysis where synonyms and paraphrases should be treated equivalently.
2. **Contextual Representations (BERT):** BERT generates different embeddings for the same word depending on its surrounding context. In Reddit comments, the same word can carry very different sentiment depending on context, for example "sick" can mean ill (negative) or impressive (positive in slang). TF-IDF cannot capture this distinction while BERT can.
3. **Sequential Modeling (BiLSTM):** The BiLSTM architecture processes text as a sequence, capturing long-range dependencies between words. This is important for longer Reddit comments where the sentiment expressed at the end of a sentence may depend on context established at the beginning. TF-IDF and Logistic Regression treat text as a bag of words with no sequential information.
4. **Local Feature Extraction (CNN):** The CNN architecture applies convolutional filters to detect local patterns and key phrases regardless of their position in the text. This positional invariance is beneficial for sentiment classification where sentiment-bearing phrases can appear anywhere in a comment.
5. **Subword Handling (FastText):** FastText represents words as bags of character n-grams, allowing it to generate meaningful embeddings for informal spellings and out-of-vocabulary words common in Reddit comments such as "gonna", "wanna", and domain-specific abbreviations. TF-IDF would treat these as entirely new unknown tokens.

6. **Imbalance Handling (Complement Naive Bayes):** The Complement Naive Bayes classifier paired with GloVe embeddings is specifically designed for imbalanced datasets. By estimating complement class probabilities, it is less susceptible to the dominance of the Positive class compared to standard classifiers.

### 6.7.3 Summary of Key Observations:

| **Observation** | **Explanation** |
|---|---|
| Improved baseline > Original baseline | Better features + tuning + ensembles |
| Advanced models > Improved baseline | Semantic embeddings capture meaning TF-IDF cannot |
| BERT models perform best | Contextual embeddings handle Reddit language best |
| Ensemble methods reduce variance | Multiple models compensate for individual errors |
| class_weight=balanced critical | Prevents bias toward dominant Positive class |

## 6.8 Summary

The baseline improvement phase demonstrated that significant performance gains can be achieved through systematic feature engineering, hyperparameter optimization, and ensemble methods, even without changing the underlying feature representation from TF-IDF. The best improved baseline model achieved an F1-Score of **0.6912**, an improvement of **+0.0692** over the original baseline of 0.6220.

However, the comparison with advanced embedding models confirms that the fundamental limitation of TF-IDF, its inability to capture semantic meaning and word relationships, remains a ceiling on performance regardless of how well the classifier is tuned. Advanced models that leverage word embeddings consistently achieve higher F1-Scores by encoding the semantic content of Reddit comments in dense, meaningful vector representations.

This finding supports the conclusion that for text classification tasks involving informal, context-dependent language such as Reddit comments, semantic embedding models are essential for achieving strong classification performance.

# 7. Conclusion

This study implemented and evaluated six sentiment classification configurations on a dataset of Reddit comments about the Israeli-Palestinian conflict, comprising an original baseline, an improved baseline, and five advanced embedding-based models.

The original baseline logistic regression with TF-IDF (unigrams, max_features=5000) achieved 62.1% accuracy and an F1-score of 0.622, serving as a solid reference. Among the advanced models, the **GloVe + CNN** performed best, reaching 70.5% accuracy and an F1-score of 0.737 for the positive class, significantly outperforming the baseline. This success is attributed to the CNN's ability to capture 5-gram sentiment patterns and the trainable embedding layer that fine-tuned GloVe vectors to the Reddit domain. The **Word2Vec + BiLSTM** also surpassed the baseline with 65.2% accuracy, benefiting from bidirectional sequential processing, though it was limited by the small corpus for training embeddings from scratch.

In contrast, three advanced models underperformed relative to the baseline:

- **BERT + XGBoost** (accuracy 57.5%) suffered from extreme overfitting (training accuracy 99.3%, test accuracy 56.4%) due to the curse of dimensionality: 384-dimensional BERT features with only 2,400 training samples allowed the XGBoost model to memorise the training data. Hyperparameter tuning and SMOTE balancing provided only marginal improvements, indicating that very high-capacity models require substantially larger datasets.
- **FastText + Random Forest** (accuracy 52.4%) and **GloVe + Complement Naive Bayes** (accuracy 47.3%) both relied on averaging word vectors to produce sentence-level embeddings. This discarded word order and syntactic structure (e.g., "not good" becomes indistinguishable from "good"), severely harming sentiment detection. Additionally, the Random Forest struggled with dense, high-dimensional features, while the Complement Naive Bayes violated its conditional independence assumption when applied to correlated embedding dimensions, resulting in strong bias toward the majority neutral class.

The baseline improvement phase demonstrated that systematic feature engineering and ensemble methods can yield substantial gains without changing the underlying

representation. Enhanced TF-IDF (bigrams, max_features=10,000, sublinear scaling) combined with GridSearchCV-tuned logistic regression raised the F1-score to 0.632. A soft-voting ensemble (Logistic Regression, SVM, Random Forest) achieved an F1-score of 0.656, and a stacking ensemble with a logistic regression meta-learner reached 0.691, outperforming all advanced models except the GloVe+CNN. This finding underscores that on small-to-medium imbalanced datasets, well-tuned bag-of-words features with ensemble methods can be highly competitive.

Key lessons from this comparative study:

1. **Dataset size is a critical constraint**. With only ~3,000 labelled comments, very high-capacity models (BERT + XGBoost) are prone to overfitting, while simpler models or those with strong regularisation (e.g., CNN with dropout) generalise better.
2. **Preserving word order and syntax is essential**. Averaging embeddings (FastText, GloVe+CNB) discards negation and phrase-level sentiment cues, leading to poor performance. Sequence-aware models (CNN, BiLSTM) are superior.
3. **Embedding fine-tuning helps**. Making the embedding layer trainable (as in GloVe+CNN and Word2Vec+BiLSTM) allows domain adaptation, improving performance over fixed embeddings.
4. **Class imbalance remains a major challenge**. Despite using class weights and imbalance-aware algorithms (ComplementNB), the minority classes (negative and positive) were often misclassified as neutral. This is especially pronounced for the positive class, which is the smallest but also the most ambiguous in political discourse due to sarcasm and indirect expression.
5. **Simple baselines should not be underestimated**. The improved stacking ensemble (F1=0.691) outperformed all advanced models except the GloVe+CNN, demonstrating that careful feature engineering and model combination can rival deep learning on modest datasets.

**Limitations and Future Work**

The primary limitations of this study are the relatively small dataset size, the inherent

linguistic complexity of political discourse, and class imbalance. Future work could focus on: (1) collecting more labelled data (ideally >10,000 comments) to enable effective fine-tuning of transformer models; (2) employing data augmentation techniques tailored to embedding spaces (e.g., back-translation, synonym replacement) to balance classes; (3) exploring pre-trained transformers fine-tuned end-to-end (e.g., BERT-base, RoBERTa) rather than using them as fixed feature extractors; (4) investigating hybrid models that combine CNN or BiLSTM with attention mechanisms; and (5) applying cost-sensitive learning or focal loss to further address class imbalance.

In conclusion, while advanced embedding models offer theoretical advantages, their practical benefits on small, noisy, imbalanced social media datasets are not automatic. A well-optimised bag-of-words model with ensemble methods remains a strong baseline, and the best results in this study were achieved by a trainable GloVe embedding layer combined with a CNN – a balanced architecture that captures local patterns without excessive capacity. These findings provide actionable insights for sentiment analysis in low-resource, domain-specific contexts.

# 8. References

GeeksforGeeks. (2026, April 14). *BERT model - NLP*. https://www.geeksforgeeks.org/nlp/explanation-of-bert-model-nlp/

GeeksforGeeks. (2026, March 19). *XGBoost*. https://www.geeksforgeeks.org/machine-learning/xgboost/

GeeksforGeeks. (2025, June 28). *XGBoost multiclass classification*. https://www.geeksforgeeks.org/machine-learning/xgboost-multiclass-classification/

Soto, M. E. (2024, October 15). Optimizing LLM classification task: BERT + XGBoost solution. *LinkedIn.* https://www.linkedin.com/pulse/optimizing-llm-classification-task-bert-xgboost-solution-soto-vi66f/

Tiwari, R. (2024, September 22). *Unlocking the Power of Sentence Embeddings with all-MiniLM-L6-v2*. Medium. https://medium.com/@rahultiwari065/unlocking-the-power-of-sentence-embeddings-with-all-minilm-l6-v2-7d6589a5f0aa